



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Corso di Laurea in Informatica

Tesi di Laurea

STUDIO E SPERIMENTAZIONE CON IL
LINGUAGGIO DI AUTORIZZAZIONI CEDAR

STUDY AND EXPERIMENTATION WITH THE
CEDAR AUTHORIZATION LANGUAGE

COSIMO MATASSINI

Relatore: *Francesco Tiezzi*

Anno Accademico 2023-2024

L'autore esprime il più sincero e profondo ringraziamento al Professor Tiezzi per il prezioso supporto e la supervisione durante tutto il percorso di redazione del lavoro. Grazie a lui è stato possibile conoscere e approfondire questa tematica e la sua disponibilità ha consentito un approccio critico e metodico.

Un ringraziamento speciale ai genitori Chiara e Massimo per lo stimolo a dare il massimo e credere sempre nelle proprie capacità.

Abstract

Con l'avvento e la diffusione di Internet si è reso necessario governare il processo di digitalizzazione. La regolamentazione degli accessi nella gestione della sicurezza è diventata uno degli aspetti fondamentali nell'utilizzo delle risorse. L'obiettivo di questo studio è l'analisi approfondita di un nuovo linguaggio open-source: Cedar, realizzato da Amazon Web Services (AWS), progettato per essere espressivo e performante rispetto ad altri linguaggi, che tendono a sacrificare aspetti importanti per poterne ottimizzare altri. Cedar presenta latenze basse, garantendo tempi di risposta estremamente veloci. La sua espressività rende le regole scritte accessibili sia ai tecnici che a chi non ha conoscenze specifiche nel settore; è possibile stabilire dei criteri all'interno di un file e delegare le richieste di accesso al motore di autorizzazioni in modo da disaccoppiare il controllo degli accessi dalla logica applicativa. Le politiche possono dunque essere scritte e analizzate in maniera indipendente dal codice dell'applicazione. Prima di procedere con le richieste di operazioni, viene effettuata una chiamata al motore di autorizzazioni di Cedar, che controlla se esse possono essere effettuate. La metodologia di progettazione è sostenuta da una filosofia di sicurezza che include la prevenzione di un consumo eccessivo di risorse. I criteri del linguaggio sono progettati per essere valutati in modo sicuro. È interessante analizzare l'aspetto open-source di Cedar, un progetto innovativo, recente e in continua evoluzione, che permette di avere un'ampia comunità di sviluppatori che possono migliorarne le funzionalità. La sua semplicità di lettura e applicazione offre infatti un elevato potenziale per sviluppi futuri.

Abstract

With the advent and widespread use of the Internet, it has become necessary to govern the digitalization process. Access regulation in the management of security has turned into one of the fundamental aspects of resource utilization. The aim of this study is to conduct an in-depth analysis of a new open-source language: Cedar, developed by Amazon Web Services (AWS), designed to be both expressive and performant compared to other languages, which often sacrifice key aspects to optimize others. Cedar offers low latency, ensuring extremely fast response times. Its expressiveness makes its rules accessible to both technical users and those without specific knowledge in this field. It allows criteria to be established within a file and access requests to be delegated to the authorization engine, thus decoupling access control from application logic. Policies can be written and analyzed independently of the application code. Before proceeding with operational requests, a call is made to Cedar's authorization engine, which checks whether the operations can be performed. The design methodology is supported by a security philosophy that includes the prevention of excessive resource consumption. The language's criteria are designed to be evaluated safely. It is particularly interesting to analyze the open-source aspect of Cedar, an innovative, recent, and constantly evolving project that allows a broad community of developers to contribute to enhancing its functionalities. Its simplicity in readability and application offers indeed significant potential for future developments.

INDICE

1	Introduzione	3
2	La sicurezza nel controllo degli accessi	5
2.1	Il controllo degli accessi	6
2.2	Tipi di Autorizzazione	7
2.2.1	Role-Based Access Control	7
2.2.2	Relationship-Based Access Control	8
2.2.3	Attribute-Based Access Control	8
2.2.4	Confronto tra le tipologie di autorizzazione RBAC, Re-BAC e ABAC	9
3	Il linguaggio Cedar	11
3.1	Caso pratico esemplificativo	12
3.2	Storia del linguaggio	13
3.3	Caratteristiche del linguaggio	15
3.4	Funzionamento	20
3.4.1	Tipi di dato e operatori	20
3.4.2	Sintassi delle politiche	24
3.4.3	File Schema	29
3.4.4	Policy Validation	38
3.4.5	File delle entità	39
3.4.6	Policy Evaluation	43
3.5	Best Practices	44
4	Esperienza applicativa	47
5	Conclusioni	63
	Bibliografia	65
A	Schema Human-Readable	67
B	Schema JSON	71
C	Il file delle entità	77

INTRODUZIONE

Con l'avvento e la diffusione di Internet, che è entrato a far parte di tutte le realtà, a partire dalle grandi aziende, alle istituzioni, fino ad arrivare ad una diffusione capillare in tutte le famiglie, si è reso necessario governare il processo di digitalizzazione che ha ormai permeato ogni aspetto della vita quotidiana.

La digitalizzazione ha reso necessaria la regolamentazione degli accessi nella gestione della sicurezza, che è diventato uno degli aspetti fondamentali nell'accesso alle risorse. Gestire le autorizzazioni e il controllo degli accessi è diventata quindi una sfida cruciale per molte organizzazioni: man mano che le applicazioni e i microservizi si sono diffusi, è risultato essenziale garantire che solo determinate persone e sistemi avessero accesso alle risorse.

La gestione di questa complessità può presentare delle difficoltà, soprattutto con la crescita di team e organizzazioni. Con l'incremento del numero di risorse e utenti, è diventato sempre più difficile gestire e implementare nuove politiche. In questo ambito la scrittura di politiche come codice diventa uno standard de facto, consentendo agli sviluppatori di elaborare dei codici che possono essere versionati, testati e distribuiti. Utilizzare un approccio dichiarativo piuttosto che imperativo rende più semplice l'espressione del risultato richiesto, poiché permette di focalizzarsi sul contenuto della politica piuttosto che sulla procedura per la sua valutazione, senza bisogno di descriverne il flusso di controllo. La programmazione imperativa prevede, invece, che lo sviluppatore scriva un insieme di istruzioni che vengono eseguite in un certo ordine, concentrandosi quindi sul "come" arrivare al risultato piuttosto che sul "cosa" permette di ottenere il risultato stesso. Nella programmazione

dichiarativa, invece, si descrivono le condizioni che regolano gli accessi e le autorizzazioni senza specificare la procedura; le esecuzioni non seguono un ordine specifico, perché tale compito è affidato al motore di valutazione delle politiche e non alla sintassi del linguaggio. Inoltre, la scrittura di politiche come codice consente di separare la loro definizione dal codice dell'applicazione vero e proprio: questo migliora notevolmente la comprensione e la facilità di scrittura e lettura delle politiche stesse.

Per tutti questi motivi, Amazon Web Services (AWS) ha realizzato un nuovo progetto open-source, il linguaggio Cedar [1]. Cedar è un linguaggio di politiche realizzato per definire, in maniera dichiarativa, permessi di accesso tramite regole. È stato progettato per essere espressivo e performante, soprattutto rispetto ad altri linguaggi. Nel codice dell'applicazione, prima di procedere con le richieste di operazioni, viene effettuata una chiamata al motore di autorizzazioni di Cedar, che controlla se queste ultime possono essere effettuate.

In questa tesi ho scelto di approfondire il linguaggio Cedar poiché presenta degli aspetti particolarmente distintivi e innovativi rispetto ad altri della stessa tipologia. Sebbene Cedar debba gestire aspetti molto complessi, la sua semplicità di lettura e applicazione ha suscitato in me interesse e curiosità, e ritengo abbia un potenziale molto elevato anche per sviluppi futuri. La presente ricerca si propone di esplorare, illustrare e sperimentare le funzionalità di Cedar.

Il resto della tesi è organizzato come segue. Il Capitolo 2 offre una panoramica relativa all'evoluzione dei sistemi di regolamentazione degli accessi, dalle origini fino alla realtà attuale. Si passa quindi nel Capitolo 3 ad un esame dettagliato delle finalità e delle caratteristiche del linguaggio Cedar. Infatti, come vedremo, Cedar si distingue per la sua estrema leggibilità e facilità di apprendimento. Il capitolo entra nello specifico del funzionamento del linguaggio e del suo motore, fornendo una guida completa su termini e concetti, sulla sintassi delle politiche e sulle procedure; per ogni elemento sarà riportato un esempio di scrittura. Nel Capitolo 4 si presenta un caso pratico di utilizzo del linguaggio Cedar che consiste nella gestione reale degli accessi di una applicazione. Infine, nel Capitolo 5 si passa alle conclusioni del lavoro; si descrivono i punti di forza del linguaggio, e al contempo si prendono in esame anche gli aspetti che potrebbero essere migliorati. Si conclude con un'analisi comparativa con altri linguaggi della stessa tipologia.

LA SICUREZZA NEL CONTROLLO DEGLI ACCESSI

La sicurezza è un aspetto fondamentale nell'informatica, un campo in continua evoluzione che si occupa di proteggere i sistemi informatici, le reti e i dati da minacce e attacchi esterni.

Il controllo degli accessi è uno degli elementi essenziali della sicurezza ed ha il compito di determinare chi può accedere alle risorse all'interno di un sistema o eseguire operazioni; permette di salvaguardare le informazioni sensibili evitando che finiscano nelle mani di utenti malintenzionati.

Gli attacchi cibernetici ai dati riservati possono avere serie conseguenze, fra le quali il furto di proprietà intellettuali (licenze, brevetti e marchi), la diffusione di dati personali e informazioni su dipendenti e clienti, fino ad arrivare alla perdita di finanziamenti aziendali [10].

Le tecniche di controllo degli accessi offrono diversi livelli di flessibilità e possono essere utilizzate in combinazione per creare un sistema di gestione degli accessi sicuro e personalizzabile. La scelta della tecnica, o delle tecniche da utilizzare, dipende dai requisiti operativi e dalle specifiche esigenze di sicurezza.

Passiamo adesso all'analisi dei concetti relativi al controllo degli accessi, con particolare attenzione alle tecniche di autorizzazione.

2.1 IL CONTROLLO DEGLI ACCESSI

I principi della sicurezza informatica nel campo del controllo degli accessi sono tre:

1. **Identificazione:** processo attraverso il quale un utente o un sistema si presentano ad un servizio; è il primo passo e serve a stabilire chi sta tentando di accedere alla risorsa.
2. **Autenticazione:** verifica dell'identità di un utente, di un sistema o di un processo. Può avvenire attraverso il sistema più comune, ovvero credenziali (username e password), autenticazione a due fattori, utilizzo delle nuove passkeys, token hardware, o altri metodi che garantiscano che l'utente sia effettivamente chi dice di essere. L'autenticazione è un elemento critico del controllo degli accessi, in quanto assicura che solo gli utenti legittimi possano accedere alle risorse.
3. **Autorizzazione:** processo che determina quali operazioni un utente autenticato può eseguire su una determinata risorsa e può includere la lettura, la scrittura, la modifica, l'eliminazione di dati o altre operazioni. L'autorizzazione è controllata da politiche di sicurezza che definiscono i diritti di accesso per ciascun utente o gruppo di utenti. Tutto ciò garantisce che gli utenti non eseguano azioni al di fuori delle loro autorizzazioni.

Il presente lavoro si focalizza sul processo di autorizzazione, dando per scontato che identificazione e autenticazione siano state effettuate correttamente ed in modo sicuro. Esso si basa sulle politiche di accesso, cioè linee guida che regolamentano l'accesso a utenti legittimi e escludono i malintenzionati.

Quando un utente tenta di accedere a una risorsa, il sistema verifica le sue credenziali e confronta i suoi diritti di accesso con le politiche di sicurezza, definite in precedenza, per negare o consentire l'accesso. In altre parole, ogni politica definisce l'esito alla domanda, che in un contesto reale corrisponde a una richiesta di autorizzazione per l'accesso a una risorsa: "L'utente X può fare l'azione Y sulla risorsa Z?". La risposta a questa domanda può essere sì, consentendo l'accesso, oppure no, negandolo.

2.2 TIPI DI AUTORIZZAZIONE

Nella gestione degli accessi, con l'incremento della complessità delle applicazioni, è necessario garantire un accesso sicuro e controllato alle risorse in maniera granulare.

A tal fine è possibile utilizzare diversi tipi di autorizzazione, ognuno con le proprie caratteristiche, vantaggi e svantaggi. I principali, supportati da Cedar, sono RBAC (Role-Based Access Control), Re-BAC (Relationship-Based Access Control) e ABAC (Attribute-Based Access Control). Oltre a questi ne esistono altri, come il Mandatory Access Control (MAC), e Discretionary Access Control (DAC), che però non verranno trattati in questo testo poiché meno comuni e non utilizzati dal linguaggio Cedar.

2.2.1 *Role-Based Access Control*

Il Role-Based Access Control (RBAC), o controllo degli accessi basato sui ruoli, è un modello che si fonda sull'assegnazione di ruoli predefiniti agli utenti, con permessi specifici associati a ciascun ruolo. Per ruolo si intende un modo di raggruppare gli utenti sulla base di determinate caratteristiche.

I ruoli vengono scelti in maniera opportuna dallo sviluppatore dell'applicazione e dovrebbero essere in linea con le responsabilità degli utenti. A questi ultimi possono essere assegnati uno o più ruoli, e i permessi associati a ciascuno di essi vengono ereditati dagli utenti che li possiedono. Questo modello semplifica la gestione degli accessi e consente di definire una sola volta i permessi assegnando ad ogni utente il relativo ruolo.

RBAC facilita la gestione degli accessi in ambienti complessi, in cui gli utenti possono avere diversi livelli di autorizzazione in base al ruolo che ricoprono.

RBAC presenta semplicità e facilità di gestione, specialmente in ambienti dove le responsabilità degli utenti sono ben definite e non cambiano frequentemente. Tuttavia, può essere limitato nei casi in cui gli utenti abbiano bisogno di autorizzazioni più granulari o laddove le relazioni tra gli utenti e le risorse siano maggiormente complesse.

2.2.2 *Relationship-Based Access Control*

Il Relationship-Based Access Control (Re-BAC), o controllo degli accessi basato sulle relazioni, definisce politiche di controllo degli accessi basate su relazioni tra soggetti.

Questo modello offre una maggiore flessibilità rispetto al precedente, consentendo di definire autorizzazioni basate su relazioni dinamiche. Il controllo in esame permette di utilizzare la transitività, ossia la propagazione delle autorizzazioni, per garantire che gli utenti possano accedere alle risorse in base alle relazioni che hanno con altri utenti o gruppi di utenti. In pratica, se la risorsa A è in relazione con la risorsa B, e quest'ultima è in relazione con la risorsa C, un utente che ha accesso alla risorsa A avrà accesso anche alla risorsa C. In questo modo è possibile gestire relazioni in strutture gerarchiche.

Re-BAC può risultare tuttavia complesso, in quanto richiede la definizione di relazioni tra gli utenti e le risorse, e può essere difficile da gestire in ambienti in cui le relazioni sono complesse o cambiano frequentemente. La numerosità delle relazioni può impattare sulle prestazioni, rallentando il processo della valutazione delle politiche.

2.2.3 *Attribute-Based Access Control*

L'Attribute-Based Access Control (ABAC), o controllo degli accessi basato sugli attributi, è un paradigma di autorizzazione che definisce le politiche di controllo degli accessi in base agli attributi specifici degli utenti, azioni e risorse.

Un attributo rappresenta una proprietà associata ad un soggetto, un'azione o una risorsa. Questo modello è utile in ambienti dinamici dove le necessità di accesso cambiano frequentemente e sono richieste condizioni più specifiche nel controllo degli accessi.

È possibile stabilire regole che gestiscono, oltre alle proprietà delle entità dell'applicazione, un contesto nelle richieste di autorizzazioni, il quale include informazioni aggiuntive come l'ora del giorno, la posizione geografica o il dispositivo utilizzato.

Le politiche di controllo degli accessi divengono più sofisticate e flessibili, condizionate da una vasta gamma di fattori.

ABAC è consigliabile quando le autorizzazioni devono essere dinamiche e basate su criteri specifici con un controllo granulare degli accessi. D'altro canto, può risultare complesso da implementare e gestire, in contesti in cui è necessario definire regole di autorizzazione dettagliate e un gran numero di attributi.

2.2.4 Confronto tra le tipologie di autorizzazione RBAC, Re-BAC e ABAC

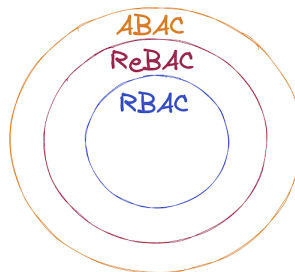


Figura 1: I tipi di autorizzazione supportati da Cedar [11].

Il linguaggio Cedar, come vedremo nel prossimo capitolo, supporta tutti e tre i tipi di autorizzazione.

RBAC permette di semplificare la gestione degli accessi in ambienti con ruoli ben definiti, senza dover valutare ogni singolo individuo; Re-BAC e ABAC offrono una maggiore flessibilità basata rispettivamente sulle relazioni e sugli attributi, consentendo di scrivere regole (policy) più sofisticate.

La facilità di implementazione e la semplice esecuzione delle operazioni di controllo degli accessi di RBAC lo rendono molto performante; in particolari contesti, tuttavia, può risultare difficoltoso da implementare, specialmente laddove siano richiesti molteplici ruoli e utenti, con gerarchie numerose.

La natura ricorsiva di Re-BAC lo rende adatto ad amministrare contesti più complessi, a scapito però delle prestazioni. Seppur più granulare

rispetto a RBAC, Re-BAC incontra difficoltà nel definire regole dipendenti da un contesto specifico.

ABAC, concepito per essere utilizzato in ambienti ancora più ampi, offre la maggiore granularità, seppur con una difficoltà di implementazione e gestione più elevata dovuta al numero rilevante di attributi e regole. In quest'ultimo caso, le prestazioni possono essere influenzate negativamente dalla complessità della sua struttura.

IL LINGUAGGIO CEDAR

Cedar è un linguaggio di politiche open-source sviluppato da Amazon Web Services (AWS).

AWS è una piattaforma di proprietà del gruppo Amazon che offre servizi di cloud computing, elaborazione e distribuzione di contenuti e molto altro, ideali per creare applicazioni sofisticate in modo flessibile, scalabile e affidabile. Lanciata nel 2002, AWS oggi copre il 58% del fatturato di Amazon facendosi strada come leader nel settore del cloud computing, offrendo oggi più di 100 servizi in cloud [5]. Cedar viene usato su larga scala in prodotti e servizi da Amazon Web Services.

Il linguaggio permette allo sviluppatore di un'applicazione di controllare gli accessi alle risorse definendo regole tanto dettagliate quanto semplici da comprendere. In pratica si tratta di un sistema di permessi.

Con Cedar è possibile stabilire dei criteri all'interno di un file e delegare le richieste di accesso al motore di autorizzazioni: in questo modo si disaccoppiano il controllo degli accessi dalla logica applicativa e le politiche possono essere scritte, modificate e analizzate in maniera indipendente dal codice dell'applicazione. Questo ha due vantaggi. Il primo consiste nella possibilità di modificare le politiche, senza dover ricompilare il codice dell'applicazione. Il secondo riguarda l'esternalizzazione delle stesse per permetterne l'utilizzo in più applicazioni.

L'approccio tradizionale invece prevede la combinazione del controllo degli accessi con la logica applicativa, presentando diversi svantaggi quali la scrittura delle politiche all'interno del codice, incrementandone le dimensioni, peggiorandone la leggibilità e la comprensione, e influenzando

negativamente nel mantenimento, poiché la variazione delle politiche implica cambiamento di codice e viceversa.

Piuttosto che integrare la logica delle autorizzazioni con quella dell'applicazione, gli sviluppatori possono scrivere le politiche in maniera completamente indipendente da essa. Cedar, infatti, permette di scrivere le policy in un file separato. Essendo un linguaggio di dominio specifico (Domain Specific Language), ossia un linguaggio di programmazione dedicato a una particolare tecnica di rappresentazione, permette di risolvere i problemi sopra descritti, e supporta la condivisione tra più applicazioni delle stesse politiche. Oltretutto permette in maniera molto più semplice di tracciare i cambiamenti delle policy e quindi, eventualmente, di tornare ad una versione precedente.

Le politiche sono scritte come regole che specificano le condizioni in base alle quali l'accesso alle risorse è permesso o negato. Esse utilizzano i comuni metodi di controllo di accesso RBAC (role-based access control), ABAC (attribute-based access control) e Re-BAC (Relationship-based access control).

La struttura di ogni politica permette loro di essere indicizzate per un'agile consultazione e una valutazione accurata. Il design supporta valutazioni veloci e scalabili in tempo reale con latenza limitata. Ciascuna è composta da tre parametri fondamentali, a cui possono essere aggiunte condizioni aggiuntive per raffinare la richiesta di autorizzazione. Questi parametri sono:

- L'utente/soggetto che deve essere autorizzato (principal)
- L'azione che questo vuole effettuare (action)
- La risorsa sulla quale si esegue l'azione (resource).

3.1 CASO PRATICO ESEMPLIFICATIVO

Per illustrare il linguaggio Cedar, nel resto del capitolo farò riferimento al seguente caso d'uso. Un utente effettua una richiesta di visualizzazione di un file "JaneDoe.jpg" contenente una foto presente nell'applicazione.

Il motore di autorizzazioni di Cedar confronta la richiesta di autorizza-

zione con le politiche definite.

Figura 2 mostra un UML sequence diagram che rappresenta le interazioni che possono aver luogo nel caso d'uso considerato. Nel caso in cui la politica determinante neghi l'autorizzazione, viene restituito un errore di accesso negato al richiedente. Se la politica determinante permette l'accesso, l'utente riceve la risorsa richiesta.

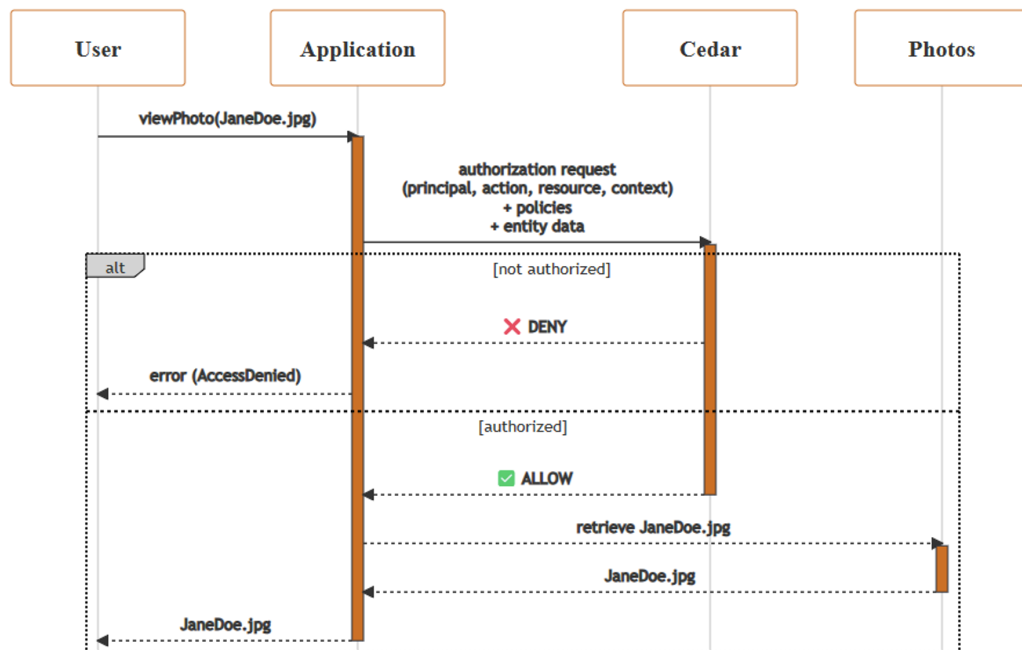


Figura 2: Diagramma di interazione tra policy e autorizzazione [13].

3.2 STORIA DEL LINGUAGGIO

In origine AWS utilizzava le politiche IAM (Identity and Access Management), ottimizzate per le applicazioni interne e inadatte per utilizzi esterni.

I linguaggi già esistenti, come OpenFGA [2] e Rego [6], eseguono le richieste di autorizzazioni sul cloud: il motore delle autorizzazioni del linguaggio risiede in server remoti, e quando si effettua una richiesta di autorizzazione, viene inviata al server remoto per essere valutata. Questo processo può essere molto lento: si deve tenere in considerazione il tempo di invio della richiesta, il tempo di ricezione della risposta e il

fatto che il server remoto debba gestire un numero elevato di richieste di autorizzazioni.

Questi linguaggi integrano la scrittura delle loro politiche all'interno delle applicazioni, rendendo difficile la manutenzione e la comprensione delle politiche stesse. La loro modifica implica il cambiamento del codice dell'applicazione e viceversa, rendendo difficile il tracciamento dei cambiamenti delle politiche e la loro condivisione tra più applicazioni.

Questi linguaggi, quando prediligono le performance, tendono a sacrificare la leggibilità delle politiche, rendendo difficile la comprensione delle stesse. I linguaggi che invece prediligono la leggibilità, sacrificano le performance.

Negli ultimi anni, si è rilevato che la complessità nell'interfaccia uomo/tecnologia è spesso un'origine del rischio per la sicurezza. In altre parole, le persone commettono errori, si confondono e fanno clic sui pulsanti per far funzionare qualcosa senza capirne il motivo [9].

Alla luce di quanto sopra, è emersa la necessità di creare un nuovo linguaggio di politiche, Cedar, che fosse più flessibile e adatto a un utilizzo esterno (in particolare che non utilizzasse costrutti AWS), progettato per prendere decisioni di autorizzazione in tempo reale e con latenze basse, senza compromettere la sicurezza e la leggibilità delle politiche.

Il nome del linguaggio, deriva dai due linguaggi utilizzati precedentemente per le politiche IAM, che iniziavano con le lettere rispettivamente "A" e "B", ed erano entrambi nomi di alberi. Cedar è stato scelto in quanto inizia con la lettera "C".



Figura 3: Logo di Cedar [1].

Il linguaggio è open source, ovvero chiunque può contribuire al suo sviluppo ed è utilizzabile gratuitamente. Questo permette di avere un'ampia comunità di sviluppatori che possono migliorare le funzionalità risolvendo eventuali bug; è possibile utilizzare il motore di autorizzazioni in locale, senza dover inviare le richieste di autorizzazione a server remoti, garantendo tempi di risposta molto più veloci. AWS fornisce il SDK (Software Development Kit), un insieme di strumenti che permettono e facilitano lo sviluppo delle applicazioni che integrano il sistema di autorizzazioni.

3.3 CARATTERISTICHE DEL LINGUAGGIO

Il linguaggio Cedar è stato progettato per essere simultaneamente espressivo, performante, sicuro e analizzabile; presenta latenze basse, garantendo tempi di risposta estremamente veloci. La sua espressività rende le regole scritte accessibili sia ai tecnici che a chi non ha conoscenze specifiche nel settore.

Come riesce a ottenere tutti questi risultati?

Le ottime prestazioni sono date dal fatto che il linguaggio è stato scritto in **Rust**, un linguaggio di programmazione ad alto livello che riesce a raggiungere le performance e la gestione efficiente della memoria del linguaggio C, noto per essere uno dei più performanti.

Il tempo necessario per valutare le richieste di autorizzazioni è molto basso; le applicazioni o i servizi che utilizzano Cedar sono responsabili del recupero delle politiche e dei dati: viene lasciata loro la possibilità di decidere il modo in cui farlo.

Quando un sistema contiene un elevato numero di criteri di autorizzazione, decidere quali di questi includere diventa una sfida interessante. Caricare tutti i criteri in memoria non è fattibile. Cedar utilizza un meccanismo per decidere quali criteri sono rilevanti per una particolare richiesta di autorizzazione e quali possono essere ignorati. Si tratta di selezionare un sottoinsieme di quelle politiche che sono effettivamente rilevanti per quella richiesta e valutare solo quelle, per permettere al linguaggio di essere effettivamente scalabile. Il problema di come selezionare quel sottoinsieme viene chiamato "policy slicing". L'utilizzo di questa tecnica

permette di incrementare le prestazioni di oltre 15 volte.

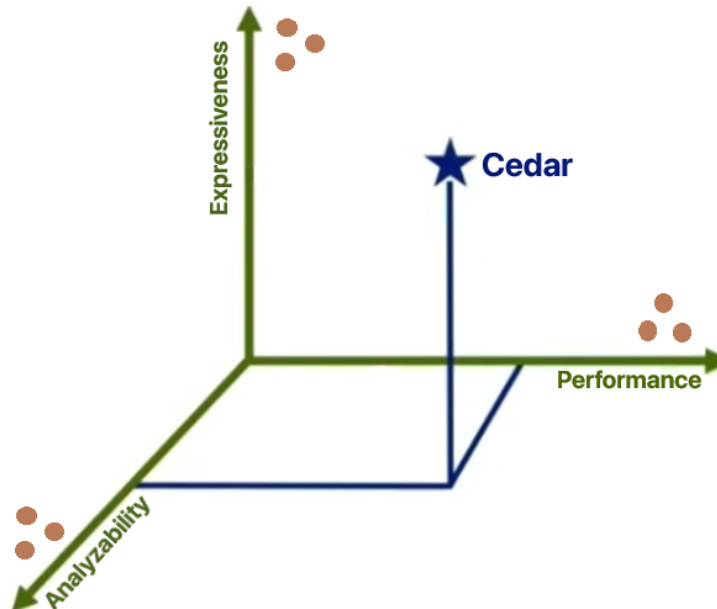


Figura 4: Confronto tra Cedar e altri linguaggi [4]

L'immagine in Figura 4 rappresenta uno spazio tridimensionale che descrive come Cedar si differenzia rispetto ad altri linguaggi per i criteri riportati di seguito. Da un lato si ha "Expressiveness", che rappresenta il livello di espressività (la capacità di descrivere le varie situazioni che si possono presentare in casi reali) del linguaggio, molto positivo se c'è il bisogno di essere flessibile. In generale, con i linguaggi di programmazione, più il linguaggio è espressivo e meno elementi si possono garantire sulle prestazioni. Ad esempio, se il linguaggio include costrutti di cicli, non è sempre possibile garantire che un programma termini. D'altra parte, per ottenere prestazioni davvero buone (lato di 'Performance', o prestazioni), si tende a limitare gli altri due fattori. La terza dimensione è caratterizzata dall'analizzabilità ("Analyzeability"). Se il linguaggio include alcune caratteristiche per essere sufficientemente espressivo, è probabile che non sia possibile analizzarle. I linguaggi già presenti in rete prediligono uno dei tre criteri, trascurando gli altri due. Cedar, invece, riesce a bilanciarli tutti e tre, equiparando o superando addirittura ogni altro linguaggio per ogni criterio.

Il motore di autorizzazioni è stato costruito tramite "verification-guided development", un processo che coinvolge i cosiddetti "automated reaso-

ning" e "differential testing".

Il ragionamento automatico è quel campo dell'informatica che tenta, tramite l'utilizzo di computer, di fornire garanzie su ciò che un sistema o un programma farà o non potrà mai fare basandosi su prove matematiche [14]. Gli strumenti di ragionamento automatico tentano di dare risposte a domande su un programma utilizzando tecniche matematiche note, verificando cosa c'è di vero in un'affermazione o in un'espressione. Con il ragionamento automatico si può dimostrare che la progettazione o l'implementazione di un sistema obbedisce alle sue specifiche e che funziona nel modo in cui è stato progettato. Vengono raggiunti questi obiettivi producendo dimostrazioni in logica formale supportate da teoremi matematici o verità note.

Quando si utilizza il ragionamento automatico, viene presentata un'enunciazione del problema, dopodiché il sistema calcola e convalida le ipotesi rispetto alla criticità presentata. Il software esegue questa operazione finché non esaurisce tutte le opzioni. In particolare, in questo contesto, il ragionamento automatico viene utilizzato per consentire alle applicazioni di resistere a tentativi di accesso indesiderati assicurando, oltre alla sicurezza, che il software funzioni come previsto e progettato. Il ragionamento automatico permette dunque di controllare la robustezza del software. Il linguaggio di programmazione utilizzato è **Dafny**.

Il "differential random testing" è una tecnica utilizzata per generare un enorme numero di input diversi in maniera casuale e per confrontare i risultati ottenuti dal modello di Dafny con quelli del codice di produzione. Se entrambe le parti producono lo stesso output, allora c'è una buona probabilità che l'implementazione corrisponda al modello. A questo scopo, viene utilizzato il framework "**cargo-fuzz**" per software scritto in Rust. Ci sono circa 10000 righe di codice in Rust rispetto alle 500 di Dafny.

Lo scopo di generare input casuali consente di considerare sia test manuali, che altrimenti non sarebbero stati presi in esame, sia di provare tutte le altre combinazioni possibili. Per questo vengono generati dati coerenti tra loro, producendo anche politiche che utilizzano i principali costrutti linguistici di Cedar.

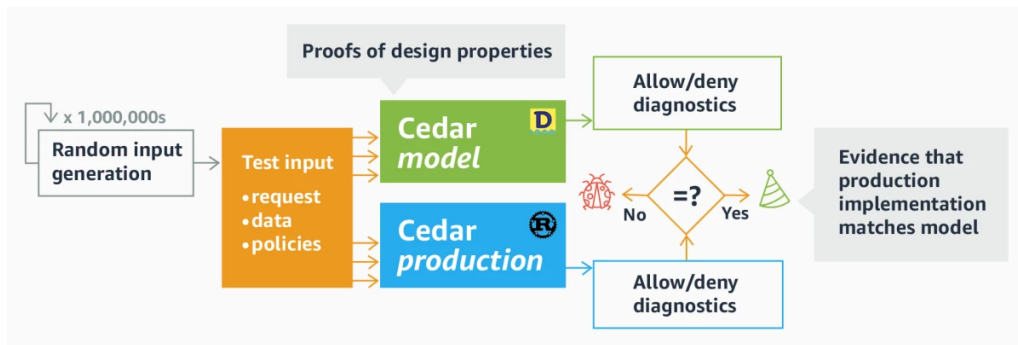


Figura 5: Automated reasoning e differential random testing [7].

La fase di differential random testing è stata implementata utilizzando il linguaggio di programmazione Rust.

Il team di AWS ha sviluppato un linguaggio che permette il refactoring (cambiamento del codice senza alterarne il comportamento, per renderlo più leggibile) delle politiche in sicurezza. Gli strumenti a disposizione assicurano che ogni cambiamento delle politiche non incrementi né diminuisca l'insieme delle azioni permesse; vengono esaminati tutti gli input possibili delle politiche, sia quelle a cui è stato fatto il refactoring che quelle nuove, dopodiché ne vengono analizzate le differenze.

Le politiche di Cedar terminano con un punto e virgola, sempre per ragioni di sicurezza. Il punto e virgola infatti protegge dagli errori di troncamento. Una politica con certe condizioni che ne raffinano il contesto potrebbe essere troncata, rimuovendo questi criteri, portando l'applicazione a effetti indesiderati (ad esempio permessi meno stringenti).

Quando una politica presenta un errore, Cedar potrebbe interrompere e negare la richiesta di autorizzazione invece di ignorare l'errore e procedere a valutare altre policy. Questo non succede, sempre per questioni di sicurezza. Se in un sistema ci fosse un numero arbitrario di politiche eseguite correttamente e l'aggiunta di una ulteriore politica alla fine contenente un errore, allora il 100% di tutte le decisioni di autorizzazione nel sistema potrebbero iniziare a fallire, semplicemente perché qualcuno ha introdotto un errore in una nuova politica.

Cedar restituisce una diagnostica che indica che qualche politica ha generato errori; è a discrezione degli sviluppatori delle applicazioni decidere, in questo caso, se terminare il programma.

Per evitare problemi è buona prassi, prima di eseguire l'applicazione, rilevare eventuali dichiarazioni di politiche che potrebbero presentare errori in fase di "policy-validation" tramite il file schema (argomento che viene trattato successivamente), in modo da essere corrette prima dell'utilizzo.

La metodologia di progettazione di Cedar è sostenuta da una filosofia di sicurezza che include la prevenzione di un consumo eccessivo di risorse. Nessun operatore Cedar può causare operazioni di blocco, cicli non terminanti o gravi riduzioni delle prestazioni. I criteri del linguaggio sono progettati per essere valutati in modo sicuro. Cedar non contiene funzioni di accesso al file system, chiamate di sistema o operatori di rete.

Le espressioni regolari e gli operatori di formattazione delle stringhe sono stati intenzionalmente omessi dal linguaggio perché il loro funzionamento contrasta con gli obiettivi di sicurezza: Cedar utilizza solo regole semplici, infatti le espressioni regolari e le operazioni di formattazione delle stringhe comportano rischi in fase di esecuzione e prestazioni scadenti. Basterebbero poche decine di concatenazioni per ampliare una semplice stringa ad una dimensione nell'ordine dei terabyte, eccedendo la memoria disponibile su quasi tutti i server web moderni. Per garantire latenze di esecuzione limitate, un motore di autorizzazione dovrebbe proteggersi da un'eccessiva allocazione di memoria e fallire durante l'esecuzione. Le espressioni regolari e gli operatori di formattazione delle stringhe, inoltre, non sono adatti alle tecniche che fanno uso di "automated reasoning".

Per esprimere concetti dove sarebbero necessari questi operatori, Cedar utilizza approcci alternativi che rimangono in linea con gli obiettivi di sicurezza. Ad esempio, Cedar include un operatore "like", che risolve molte situazioni che altrimenti richiederebbero espressioni regolari.

Per quanto riguarda i numeri decimali, Cedar utilizza il tipo Decimal con una precisione fissa, in cui il numero massimo di cifre dopo la virgola è quattro. La maggior parte degli altri linguaggi di programmazione, utilizzano la rappresentazione dei numeri in virgola mobile floating-point, che consente valori più grandi o più piccoli con diversi gradi di precisione. Il motivo di questa differenza è che i numeri in virgola mobile per loro natura sono imprecisi e l'imprecisione non è una proprietà desiderabile di un sistema di autorizzazione. Facendo uso di un'aritmetica finita, non tutti i numeri hanno una rappresentazione binaria precisa. Da questo

principio deriva il fatto che non siano supportati neanche operatori di moltiplicazione e divisione per numeri decimali, che porterebbero anch'essi a risultati indesiderati.

3.4 FUNZIONAMENTO

Vediamo adesso come si scrivono le politiche di Cedar. Oltre alle politiche di autorizzazione, che ribadiamo permettono di descrivere chi è autorizzato a eseguire determinate azioni su determinate risorse ed in quale contesto, sono presenti due ulteriori elementi che vengono approfonditi di seguito.

- **Schema:** il file che definisce la struttura degli oggetti presenti all'interno dell'applicazione. La struttura gerarchica, il formato delle relazioni e degli attributi degli oggetti vengono definiti nello schema per la validazione delle politiche.
- **Entità:** il file che definisce le entità. I soggetti, le azioni e le risorse che fanno parte dell'applicazione sono tutte rappresentate come entità.

3.4.1 *Tipi di dato e operatori*

I tipi di dato supportati da Cedar sono sette:

- **Boolean:** il valore può essere `true` (vero), o `false` (falso)
- **Long:** un numero intero nell'intervallo $[-2^{63}, 2^{63} - 1]$. Nel caso in cui venga eseguita un'operazione che supera gli estremi dell'intervallo, Cedar restituisce un errore di overflow
- **String:** una sequenza di caratteri racchiusa tra doppie virgolette, ad esempio `"Ciao"`
- **Entity:** gli elementi `principal`, `action` e `resource` sono entità dell'applicazione, contraddistinte con lo UID (Unique Identifier), che li identifica in maniera univoca. Lo UID è composto dal tipo e dal

nome dell'entità (racchiuso tra virgolette), separati da ":". Le entità vengono descritte in maniera approfondita in seguito.

- **Set:** insieme di elementi raggruppati utilizzando le parentesi quadre. Gli elementi sono separati tra di loro da una virgola. L'insieme stesso viene considerato come un elemento unico, quindi è possibile ricorsivamente inserire un set all'interno di un altro set. Gli elementi contenuti nei set possono contenere dati non necessariamente dello stesso tipo (detti omogenei). `[]` è un esempio di set vuoto e `[12, true, "Ciao"]` è un esempio di set contenente un valore di tipo Long, uno di tipo Boolean e uno String
- **Record:** insieme di elementi (in questo caso detti attributi) raggruppati utilizzando le parentesi graffe. Ogni attributo consiste in un nome, sottoforma di String, e un valore che può essere di ogni altro tipo di dato; questi sono separati da ":". Un esempio di attributo è `"name" : "Cosimo"`, che prende come valore un dato di tipo String. Gli attributi all'interno di un record sono separati tra di loro tramite la virgola. Come per i set, i record possono essere annidati. Un esempio di record contenente due attributi è `{"name" : "Cosimo", "age" : 22}`. Per accedere ad un attributo di un record e ricavarne il valore, si utilizza la "dot notation" con il nome dell'attributo in questione, ad esempio `record.name` restituisce "Cosimo"
- **Extension:** valori particolari che devono essere utilizzati chiamando un metodo (detto costruttore) che riceve come argomento il valore di interesse sottoforma di String. Al momento, Cedar supporta due extension: `decimal` e `ipaddr`. `Decimal` rappresenta un valore decimale nell'intervallo $[-\frac{2^{63}}{10^4}, \frac{2^{63}-1}{10^4}]$ con una precisione di al massimo quattro cifre decimali. Come per il tipo Long, Cedar restituisce un errore di overflow nel caso in cui venga eseguita un'operazione che supera gli estremi dell'intervallo. Un esempio di extension `decimal` è `decimal("1234.5678")`. `ipaddr` rappresenta un indirizzo IP, che può essere scritto in formato IPv4 o IPv6 utilizzando la notazione CIDR. Il costruttore deve essere scritto con `ip` e non `ipaddr`. Un esempio di indirizzo IP è `ip("192.168.1.1/16")`

Operatori supportati da Cedar:

I seguenti operatori si utilizzano in questo modo: `operando1 operatore operando2`, salvo eccezioni specificate. Gli operandi delle varie operazioni

possono essere valori o espressioni che danno come risultato un valore del tipo supportato dall'operando.

- `==` ritorna il valore `true` se i due operandi sono dello stesso tipo e hanno lo stesso valore, `false` in caso contrario
- `!=` ritorna il valore `false` se i due operandi sono dello stesso tipo e hanno lo stesso valore, `true` in caso contrario
- `<` operatore per dati di tipo `Long` che ritorna il valore `true` se il primo operando è minore stretto del secondo, `false` in caso contrario
- `>` operatore per dati di tipo `Long` che ritorna il valore `true` se il primo operando è maggiore stretto del secondo, `false` in caso contrario
- `<=` operatore per dati di tipo `Long` che ritorna il valore `true` se il primo operando è minore o uguale al secondo, `false` in caso contrario
- `>=` operatore per dati di tipo `Long` che ritorna il valore `true` se il primo operando è maggiore o uguale al secondo, `false` in caso contrario
- `+` operatore per dati di tipo `Long` che ritorna la somma dei due operandi
- `-` operatore per dati di tipo `Long` che ritorna la differenza tra il primo operando e il secondo
- `*` operatore per dati di tipo `Long` che ritorna il prodotto dei due operandi
- `&&` operatore logico per dati di tipo `Boolean` che ritorna il valore `true` se entrambi gli operandi restituiscono `true`, `false` in caso contrario
- `||` operatore logico per dati di tipo `Boolean` che ritorna il valore `false` se entrambi gli operandi restituiscono `false`, `true` in caso contrario
- `!` operatore logico unario che, anteposto ad un dato che ritorna un valore `Boolean`, inverte il suo valore.
- `if` operatore della forma `if Boolean then A else B`, dove `Boolean`

indica un valore o un'espressione che ritorna un valore di tipo Boolean, A e B dati generici o espressione di qualunque tipo. Se Boolean è true, l'operatore restituisce A, in caso contrario restituisce B

- `in` operatore per dati di tipo Entity che ritorna il valore true se il primo operando è un discendente della gerarchia del secondo operando, espressione che viene valutata come Entity o un Set di Entity. Per questo operatore valgono la proprietà transitiva (date le entità A, B, C, se A `in` B restituisce il valore true e B `in` C restituisce il valore true, allora anche B `in` C restituisce il valore true) e riflessiva (ogni entità è sempre un discendente di sé stesso: data A entità, A `in` A ritorna il valore true).
- `has` operatore per dati di tipo Record o Entity che ritorna il valore true se il primo operando contiene un attributo il cui nome è quello specificato dal secondo operando, false in caso contrario.
- `is` operatore per dati di tipo Entity che ritorna il valore true se l'entità del primo operando è del tipo specificato nel secondo operando, false in caso contrario.
- `like` operatore per dati di tipo String che ritorna il valore true se il primo operando è una stringa che contiene la stringa specificata come secondo operando, false in caso contrario. Il secondo operando deve essere una stringa letterale e può contenere il carattere speciale *, che rappresenta una sequenza di zero o più caratteri arbitrari. Per rappresentare quindi il carattere asterisco nella stringa è necessario utilizzare *.

Gli operatori di seguito sono della forma `operando1.operatore(operando2)`

- `contains` operatore per dati di tipo Set che ritorna il valore true se il primo operando contiene al suo interno il valore specificato come secondo operando, qualunque tipo esso sia, false in caso contrario
- `containsAll` operatore per dati di tipo Set che ritorna il valore true se il primo operando contiene al suo interno tutti i valori specificati nel Set come secondo operando, false in caso contrario
- `containsAny` operatore per dati di tipo Set che ritorna il valore true se il primo operando contiene almeno uno dei valori specificati nel

Set come secondo operando, false in caso contrario

- `lessThan` come `<`, ma per dati di tipo `Extension decimal`
- `lessThanOrEqual` come `<=`, ma per dati di tipo `Extension decimal`
- `greaterThan` come `>`, ma per dati di tipo `Extension decimal`
- `greaterThanOrEqual` come `>=`, ma per dati di tipo `Extension decimal`
- `isInRange` operando per dati di tipo `Extension ipaddr` che ritorna il valore `true` se il primo operando è un indirizzo IP che rientra nell'intervallo di indirizzi specificato dal secondo operando, false in caso contrario

I seguenti operatori sono unari, della forma `operando.operatore()`

- `isIpv4` operatore per dati di tipo `Extension ipaddr` che ritorna il valore `true` se l'operando è un indirizzo IP in formato IPv4, false in caso contrario
- `isIpv6` come `isIpv4`, ma per indirizzi IP in formato IPv6
- `isLoopback` operatore per dati di tipo `Extension ipaddr` che ritorna il valore `true` se l'operando è un valido indirizzo IP di loopback, false in caso contrario
- `isMulticast` operatore per dati di tipo `Extension ipaddr` che ritorna il valore `true` se l'operando è un valido indirizzo IP multicast, false in caso contrario

3.4.2 Sintassi delle politiche

Le politiche di Cedar vengono scritte in un file dedicato `.cedar` separato dal codice dell'applicazione. Le politiche sono indipendenti tra di loro, hanno una struttura ben definita e vengono separate dal carattere "punto e virgola" (`;`).

È possibile inserire commenti all'interno dei file delle politiche, in una posizione qualunque, preceduti da `//`. Essi servono solo come

documentazione o appunti; dal punto in cui viene inserito il commento fino alla fine della riga, il testo non viene considerato.

Ogni politica viene definita dall'effetto che questa produce: "permit", cioè permettere l'accesso a ciò che viene definito all'interno della politica, oppure "forbid", cioè negarlo. Le politiche forbid servono per manifestare in maniera esplicita la negazione di alcune richieste di autorizzazione, garantendo che vengano sempre negate, indipendentemente dalle altre politiche, anche quelle che verranno scritte in futuro.

All'interno delle politiche soddisfatte, ovvero quelle che valutano una richiesta di autorizzazione (si tenga presente che una politica si definisce soddisfatta sia che autorizzi o meno la richiesta), si distingue la politica determinante, che decide il comportamento finale dell'autorizzazione, ed è quella che prevale su tutto.

In caso di molteplici politiche soddisfatte che abbiano un comportamento diverso (permesso o negazione), quelle di negazione acquisiscono predominanza. Vale a dire, la presenza di almeno una politica soddisfatta con effetto "forbid" determina che l'accesso venga negato, indipendentemente dalle altre politiche soddisfatte con effetto "permit".

L'ambito (chiamato scope) viene specificato all'interno di parentesi graffe ed è composto da 3 elementi: l'utente (principal), l'azione (action) e la risorsa (resource), separati ognuno da una virgola. In questa parte è possibile specificare le condizioni che utilizzano la strategia RBAC (Role-Based Access Control).

A partire dal seguente esempio, tutte le volte che si incontra "... " si intende che è presente un blocco di codice che non viene riportato per brevità e che in quel contesto non è rilevante.

```
//Struttura base di una politica con effetto "Allow"
```

```
permit (  
    principal,  
    action,  
    resource  
);
```

```
//Struttura base di una politica con effetto "Deny"
```

```
forbid (  
    principal,
```

```

    action,
    resource
);

```

PRINCIPAL Principal rappresenta un utente, un ruolo, un gruppo o una qualsiasi entità che può essere autorizzata o meno (a seconda dell'effetto della politica) ad eseguire ciò che viene indicato in action su ciò che è specificato in resource. Principal può essere seguito dagli operatori ==, is o in. is e in possono essere usati insieme, purché nel rispettivo ordine; ad esempio `principal is User in UserGroup::"group1"`. Se in una politica non viene specificato il principal, la politica si applica a qualsiasi principal presente nell'applicazione, senza distinzioni. Un esempio di controllo sul principale che una politica potrebbe includere è `principal == User::"Cosimo"`. In questo caso la politica viene valutata se il principal è di tipo "User" (utente in italiano), e il suo nome è Cosimo.

ACTION Action rappresenta l'azione, o l'insieme di azioni, che il principal può eseguire su ciò che viene specificato in resource. Come per principal, se non viene specificata un'azione, la politica si applica a qualsiasi azione presente nell'applicazione, senza distinzioni. Action può essere seguito dagli operatori == o in. A differenza di principal e resource, che supportano solo entità con in, action può utilizzare questo operatore con un set. Due esempi di controllo sull'azione che una politica potrebbe includere sono `action == Action::"readDocument"` e `action in [Action::"deleteDocument", Action::"writeDocument"]`. In questo caso la prima politica viene valutata se l'azione richiesta è "readDocument", di tipo Action, mentre la seconda politica viene valutata se l'azione richiesta è "deleteDocument" o "writeDocument", di tipo Action.

RESOURCE Resource rappresenta la risorsa, o l'insieme di risorse, su cui il principal può eseguire l'azione specificata in action. Gli operatori utilizzabili con resource sono gli stessi di principal. Se non viene specificata una risorsa, la politica si applica a qualsiasi risorsa presente nell'applicazione, senza distinzioni. Un esempio di controllo sulla risorsa che una politica potrebbe includere è `resource is Document`. In questo caso la politica viene valutata se la risorsa richiesta è di tipo Document, qualunque essa sia.

Se necessario, Cedar permette di specificare ulteriori condizioni che influiscono sulla valutazione della politica. Le clausole "when" e "unless", seguite da un blocco di condizioni, raffinano la richiesta di autorizzazione attraverso le strategie Re-BAC (Relationship-Based Access Control) e ABAC (Attribute Based Access Control).

In particolare, la clausola "when" permette di specificare le condizioni che devono essere soddisfatte affinché la politica venga valutata come soddisfatta, mentre "unless" permette di specificare le condizioni che non devono essere soddisfatte affinché la politica venga valutata come soddisfatta.

```
//Struttura di una politica che presenta la clausola "when"
```

```
permit (  
    principal,  
    action,  
    resource  
)  
when {  
    ...  
};
```

```
//Struttura di una politica che presenta la clausola "unless"
```

```
permit (  
    principal,  
    action,  
    resource  
)  
unless {  
    ...  
};
```

```
//Struttura di una politica che presenta sia la clausola "when" che  
    la clausola "unless"
```

```
permit (  
    principal,  
    action,  
    resource  
)  
when {  
    ...  
}
```

```
unless {
    ...
};
```

Dall'ultima politica, vediamo che è possibile utilizzare sia la clausola "when" che la clausola "unless" all'interno della stessa politica, che quindi viene valutata come soddisfatta solo se le condizioni specificate all'interno della clausola "when" sono soddisfatte e le condizioni specificate all'interno della clausola "unless" non sono soddisfatte. Niente vieta di utilizzare solo una delle due, con le condizioni "negate" dell'altra clausola. Tuttavia, questo porterebbe ad una peggiore leggibilità del codice; questo per precisare che le due clausole non hanno utilizzi differenti, ma servono per facilitare la scrittura e la comprensione delle politiche.

In maniera opzionale, ogni politica può essere preceduta da annotazioni, che non hanno un impatto sulla sua valutazione, ma possono risultare utili per servizi esterni. Queste sono scritte sottoforma della seguente stringa: "@annotationname("annotation value")", dove "annotationname" indica il nome dell'annotazione e "annotation value" indica la stringa che descrive l'annotazione stessa. Si precisa che le annotazioni fanno parte delle politiche e che quindi possono essere utilizzate ad esempio come tag o per descriverne il comportamento.

Oltre alle politiche che abbiamo appena visto, è possibile utilizzare dei template: la loro struttura è la stessa di quelle "classiche", con la differenza che, invece di specificare il principal o la resource, è possibile inserire dei segnaposto (placeholders). Essi vengono utilizzati per rendere generiche le politiche, in modo da poterle riutilizzare in contesti diversi. I segnaposto vengono anteposti al secondo operando utilizzando il carattere ? dell'operazione che può essere == o in. Un esempio di "policy template" è il seguente:

```
permit (
    principal in ?principal,
    action,
    resource == ?resource
);
```

I "policy template" non sono utilizzabili. È necessario creare politiche basate su di essi specificando il principal e la resource. Le politiche create

in questa maniera sono dinamiche: quando viene cambiato il codice di un template, tutte le politiche associate a questo vengono aggiornate di conseguenza. I template possono ridurre di molto le prestazioni dell'applicazione, ma sfruttando l'algoritmo di policy-slicing, la differenza è trascurabile.

3.4.3 *File Schema*

Lo schema è l'elemento del linguaggio che definisce i tipi di entità (la loro struttura) riconosciuti dall'applicazione. La sua definizione non è strettamente necessaria per il funzionamento delle politiche di autorizzazione, ma è fortemente consigliata per effettuare la "policy-validation", in modo da garantire la correttezza delle politiche stesse.

Prima di procedere con la definizione dello schema, è necessario introdurre il concetto di namespace: un "contenitore" che permette di raggruppare entità logiche correlate in un blocco unico per evitare conflitti tra nomi. In altre parole, i namespace forniscono un modo per organizzare il codice in modo logico e gerarchico. Ad esempio, se nell'applicazione fossero presenti due entità con lo stesso nome che rappresentano concetti diversi, Cedar non sarebbe in grado di distinguerle. In questo caso, utilizzando due namespace per i due contesti, si evita il conflitto. Quando si fa riferimento ad un'entità presente in un namespace diverso da quello in cui è stata definita, è necessario utilizzare il suo UID comprendendo anche il nome del namespace di cui fa parte. Nel caso in cui si fa riferimento ad entità all'interno dello stesso namespace in cui queste vengono definite non è obbligatorio specificarne il nome.

Ci sono due modi di definire uno schema: in formato JSON (JavaScript Object Notation), e "Human-Readable". Quest'ultimo è consigliato da Cedar.

Formato JSON

Il file schema definito con JSON richiede un oggetto racchiuso tra parentesi graffe la cui definizione contiene uno o più namespace contenenti due oggetti: `entityTypes` e `actions`; può essere incluso in maniera

opzionale il terzo oggetto `commonTypes`. I namespace sono separati tra di loro con una virgola e la loro definizione è racchiusa tra parentesi graffe.

```
{
  "namespace1": {
    "entityTypes": { ... },
    "actions": { ... },
    "commonTypes": { ... }
  },
  ...,
  "namespaceN": {
    "entityTypes": { ... },
    "actions": { ... }
  }
}
```

`EntityTypes` è la parte in cui viene definita la struttura di tutti i principali e resource supportati nell'applicazione. È un oggetto contenente un numero arbitrario di definizioni, separate tra di loro con una virgola, ognuna come segue:

```
"EntityType": {
  "memberOfTypes": ["parentType1", ..., "parentTypeN"],
  "shape": { ... }
}
```

`EntityType` indica il nome del tipo dell'entità, `memberOfTypes` specifica un set di identificativi di entità, `parentType1`, ..., `parentTypeN`, (se queste appartengono ad un namespace diverso da quello in cui siamo, è necessario specificarlo) che possono essere ascendenti nella gerarchia dell'istanza dell'entità (se il set è vuoto, il campo `memberTypes` può essere omissso) e `shape` contiene un record contenente tutti i tipi degli attributi (se non ci sono attributi, il campo `shape` può essere omissso).

```
"shape": {
  "type" : "Record",
  "attributes": {
    "attributeName1": {
      "type": "attributeType1"
    },
    ...
  }
}
```

```

    ...,
    "attributeNameN": {
        "type": "attributeTypeN"
    }
}

```

I vari `attributeName1`, ..., `attributeNameN` indicano i nomi degli attributi e `attributeType` il loro tipo. La loro presenza è richiesta di default, ma è possibile specificarne l'opzionalità aggiungendo `"required": false` nella descrizione dell'attributo.

```

"attributeName": {
    "type": "attributeType",
    "required": false
}

```

Tutti i tipi di dato di Cedar sono supportati; se un attributo è un record, questo deve essere definito come sopra, cioè il campo `"type"` deve essere specificato come `"Record"` e il campo `"attributes"` deve contenere la struttura del record. Se un attributo è un set, il campo `"type"` deve essere specificato come `"Set"` e deve essere incluso anche il campo `"element"` che specifica il tipo degli elementi al suo interno.

```

"AttributeName": {
    "type": "Set",
    "element": {
        "type": "AttributeType"
    }
}

```

Se un attributo è un entità (che deve essere definita nel file) deve essere incluso anche il campo `"name"` che identifica il suo tipo; si veda ad esempio l'entità `"Proprietario"` di tipo `Utente`:

```

"Proprietario": {
    "type": "Entity",
    "name": "Utente"
}

```

Allo stesso modo, se un attributo è di tipo `Extension`, deve essere

specificato il campo "name" che ne identifica il tipo (ipaddr o decimal):

```
"AttributeName": {
  "type": "Extension",
  "name": "ipaddr"
}
```

oppure

```
"AttributeName": {
  "type": "Extension",
  "name": "decimal"
}
```

Actions è la parte in cui vengono definite le azioni. Ne contiene un numero arbitrario separate una dall'altra da una virgola e racchiuse tra parentesi graffe. La struttura di ogni azione è la seguente:

```
"ActionName": {
  "memberOf": [ParentName1, ..., ParentNameN],
  "appliesTo": {
    "principalTypes": ["PrincipalEntityType1",
      ..., "PrincipalEntityTypeN"],
    "resourceTypes": ["ResourceEntityType1",
      ..., "ResourceEntityTypeN"],
    "context": { ... }
  }
}
```

ActionName indica il nome dell'azione, memberOf specifica un set di azioni ascendenti nella gerarchia dell'azione stessa. Anche in questo caso, se il set è vuoto il campo memberOf può essere omissso. I vari ParentName1, ..., ParentNameN devono essere della forma che segue

```
{
  "id": "ActionParentName"
}
```

oppure, se l'azione è definita in un namespace diverso da quello in cui siamo, è necessario specificare anche il tipo nel campo "type":

```
{
  "id": "ActionName",
  "type": "Namespaces::Action"
}
```

`appliesTo` indica un set di oggetti contenenti gli identificativi dei tipi di `principal` e `resource` a cui si può applicare l'azione. Il tipo delle azioni (actions) è sempre "Action" e non è necessario specificarlo nel file.

Se i set di `principalTypes` e `resourceTypes` sono vuoti non è possibile utilizzare l'azione. Questo avviene quando la si vuole utilizzare solamente come `parent`.

Se `principalTypes` o `resourceTypes` vengono omessi dalla clausola `appliesTo`, si può rimuovere direttamente la clausola e l'azione può essere eseguita solo su entità non specificate (None nella richiesta di autorizzazione come `principal` o `resource`).

`context` è la parte che definisce gli attributi che possono essere presenti nel campo `context` di una richiesta di autorizzazione che utilizzano l'azione. La sua struttura è esattamente la stessa del campo `shape` in `EntityTypes`.

`CommonTypes` è la sezione in cui è possibile definire tipi la cui struttura viene utilizzata più volte nello schema (per entità che hanno molti attributi in comune). In questo modo si evita la ridondanza del codice e si rende più leggibile il file schema. La definizione di entità comuni in questa sezione permette di modificarne la struttura in un solo punto (applicandola a tutte le entità che la utilizzano), senza dover modificare ogni singola occorrenza nel file.

```
"commonTypes": {
  "CommonType1": { ... },
  ...,
  "CommonTypeN": { ... }
}
```

I vari `CommonType1`, ..., `CommonTypeN` indicano i nomi dei `CommonType` e il loro contenuto è la specifica di attributi esattamente come nel campo `attributes` di `shape` in `EntityTypes`. I tipi comuni possono essere utilizzati sia da attributi nel campo `context` di `appliesTo` in `actions` che da

principal e resource quando si deve specificare il valore del campo "type". I commonTypes possono essere utilizzati a loro volta da altri commonTypes, facendo attenzione a non creare cicli (ad esempio un commonType A contiene un attributo di tipo B, anch'esso un commonType, e B contiene un attributo di tipo A). Cedar è in grado di riconoscere questo tipo di problema e restituire, nel caso, un errore.

```
"Attribute1": {
  "type": "CommonType"
}
```

Attribute1 fa riferimento ad un commonType definito nel campo commonTypes, la cui definizione è riportata di seguito.

```
"commonTypes": {
  ...,
  "CommonTypeName": {
    "type": "Record",
    "attributes": {
      ...,
      "attributeName": {
        "type": "attributeType"
      }
    },
    ...
  }
},
...
```

Formato Human-Readable

Il file schema definito come "Human-Readable" (con estensione .cedar-schema) richiede in maniera opzionale un numero arbitrario di namespace contenenti gli elementi entity (per definire la struttura di principal o resource), action (per definire la struttura di action) o type (per definire i tipi comuni come visto sopra in commonTypes) in ordine randomico. Le regole del funzionamento sono le stesse di quelle dello schema in formato JSON; se un'entità appartiene ad un namespace diverso da quello in cui viene definita, è necessario specificare il nome del namespace di

appartenenza.

La definizione dei namespace è la seguente:

```
namespace Namespace1Name {
    ...
};
...;
namespace NamespaceNName {
    ...
};
```

Vediamo adesso come si definiscono gli elementi all'interno di un namespace.

entity specifica una o più entità (principal o resource) che hanno la stessa struttura:

```
entity Entity1Name, ..., EntityNName in [Parent1Name, ...,
    ParentNName] {
    ...
};
```

Entity1Name, ..., EntityNName sono i nomi delle entità, Parent1Name, ..., ParentNName sono i nomi delle entità ascendenti nella gerarchia dell'istanza dell'entità (se non ci sono entità ascendenti, il campo in può essere omissso e se l'entità è una sola le parentesi quadre possono essere rimosse).

All'interno delle parentesi graffe vengono specificati gli attributi dell'entità nel seguente modo:

```
{
    attribute1Name: attribute1Type,
    ...,
    attributeNName: attributeNType
}
```

Se vogliamo definire un attributo come opzionale basta inserire "?" dopo il suo nome: attributeName?: attributeType

Se l'entità non presenta attributi (è vuota), le parentesi graffe possono essere omesse.

I tipi di dato supportati dagli attributi sono tutti quelli di Cedar. Se un attributo è di tipo Boolean, deve essere specificato `Bool` in `attributeType`. Se un attributo è un record, la sua definizione è la seguente:

```
...
attributeName: {
  attributeType1: attributeType1Type,
  ...,
  attributeTypeN: attributeTypeNType
}
...
```

Se un attributo è un set, la sua definizione è la seguente:

```
...
attributeName: Set<attributeType>
...
```

Se un attributo è un'entità, deve essere specificato il suo identificativo (UID) in `attributeType`. Se un attributo è di tipo Extension deve essere specificato solo il suo tipo (`ipaddr` o `decimal`) in `attributeType`.

`action` specifica una o più azioni che hanno la stessa struttura:

```
action Action1Name, ..., ActionNName in [Parent1Name, ...,
  ParentNName] appliesTo {
  ...
};
```

`Action1Name`, ..., `ActionNName` sono i nomi delle azioni, `Parent1Name`, ..., `ParentNName` sono i nomi delle azioni ascendenti nella gerarchia dell'azione (se non ci sono azioni ascendenti il campo `in` può essere omesso e se l'azione è una sola le parentesi quadre possono essere rimosse).

All'interno delle parentesi graffe vengono specificati i tipi di `principal` e `resource` a cui si può applicare l'azione e il contesto della richiesta di autorizzazione:

```
{
  principal: [Principal1Name, ..., PrincipalNName],
  resource: [Resource1Name, ..., ResourceNName],
  context: {
    ...
  }
}
```

Principal1Name, ..., PrincipalNName sono i nomi delle entità a cui si può applicare l'azione come principal, Resource1Name, ..., ResourceNName sono i nomi delle entità a cui si può applicare l'azione come resource e context è la struttura del contesto della richiesta di autorizzazione. Tutti e tre i campi possono essere omessi e si applicano le stesse regole di appliesTo viste per il formato JSON. Se nel campo principal o resource è presente un solo elemento, le parentesi quadre possono essere omesse. Il campo context contiene la definizione di un record come visto per gli attributi delle entità.

type specifica un tipo comune che può essere utilizzato da più entità:

```
type TypeName = {
  ...
};
```

Al suo interno vengono specificati gli attributi del tipo comune come visto per gli attributi delle entità. Come nella definizione nel formato JSON, si deve fare attenzione a non creare cicli.

```
...
Attribute1: TypeName
...
```

Attribute1 fa riferimento a TypeName la cui definizione è riportata di seguito.

```
type TypeName = {
  attributeName: attributeType
}
```

3.4.4 *Policy Validation*

La "policy validation" è il processo che si occupa di controllare che le politiche siano state scritte correttamente senza inconsistenze in base a come è stato definito il file schema. Eseguire la validazione delle politiche permette di evitare errori prima che possano avere un impatto negativo durante la valutazione delle richieste di autorizzazione. È buona prassi aggiornare il file schema ed effettuare la validazione delle politiche ogni qualvolta vengono cambiate, in modo da assicurarsi che nessuna modifica abbia ripercussioni negative in futuro. I controlli effettuati riguardano il riconoscimento dei nomi dei tipi delle entità (ad esempio per errori di battitura), gli attributi di cui sono composte e i loro tipi, e le loro gerarchie. Inoltre vengono controllate quali azioni vengono permesse su quali tipi di utenti, risorse e contesti, o un utilizzo non corretto degli operatori (ad esempio vengono utilizzati operandi il cui tipo non è supportato dagli operatori). Da evidenziare che i set, nonostante possano contenere elementi eterogenei, le politiche vengono validate solo se tutti i valori all'interno dei set sono non-vuoti e dello stesso tipo.

Le tipologie di errore che la validazione non è in grado di riconoscere sono quelli che si verificano durante l'esecuzione dell'applicazione, quando non è possibile prevedere quali entità verranno utilizzate durante l'esecuzione, ovvero:

- **Errori di overflow:** eseguendo operazioni su dati di tipo Long il cui risultato restituisce un valore che supera i limiti, Cedar restituisce un errore. Il team di Cedar sta lavorando per risolvere questo problema, per evitare questo tipo di errore effettuando la validazione. Supponiamo di avere una politica che comprenda la somma di due attributi di entità. La politica, se scritta correttamente in base al file schema, viene validata. Durante una richiesta di autorizzazione, questa politica viene scelta dall'algoritmo di policy-slicing per essere valutata (è quindi rilevante per il contesto). Le due entità hanno due attributi di tipo Long che, quando sommati, superano il valore massimo consentito.
- **Errori di riconoscimento di Extension:** è possibile riscontrare l'utilizzo di un dato di tipo Extension non ben-formato quando non è specificato direttamente. Laddove vengano passati direttamente, Cedar è in grado di riconoscerli ed effettuarne la validazione.

Supponiamo di avere una politica che utilizza un valore `Extension decimal`. Questo valore viene definito tramite una stringa (che deve essere ben formata, in modo che Cedar la possa trasformare nel corrispondente numero decimale). Utilizzando un'attributo `String` di un'entità come parametro sul costruttore, la politica non presenta errori e viene quindi validata. L'attributo dell'entità viene successivamente definito, ma in maniera che non sia un valore `decimal` ben formato. Durante una richiesta di autorizzazione, se la politica viene scelta dall'algoritmo di `policy-slicing` per essere valutata, Cedar non riesce a generare in maniera corretta il valore dell'attributo dell'entità in `Extension decimal` e restituisce per questo motivo errore. Lo stesso criterio si applica a valori di tipo `Extension ipaddr`.

- **Errori di mancanza di entità:** se durante una richiesta di autorizzazione viene fornita un'entità non presente nel file delle entità, qualsiasi tentativo di accesso ad un suo qualsiasi attributo restituisce un errore. La validazione non è chiaramente in grado di prevedere quali sono le entità che verranno utilizzate durante l'esecuzione dell'applicazione, quindi non è in grado di riconoscere questi errori. Supponiamo di avere una politica che utilizza un qualsiasi attributo di una certa entità definita nello schema. La politica, se scritta correttamente, viene validata. Consideriamo il caso in cui questa entità non sia stata definita nel file delle entità, e che venga eseguita una richiesta di autorizzazione contenente l'identificativo di una entità dello stesso tipo di quella definita nello schema (la richiesta è lecita). La politica cerca quindi di accedere ad un attributo di un'entità che non è stata definita, e quindi restituisce errore.

3.4.5 *File delle entità*

Le entità sono i dati dell'applicazione che Cedar usa per definire i soggetti e le risorse. Il motore di autorizzazioni deve avere accesso a tutti gli attributi di queste entità, in modo da poter raccogliere le informazioni necessarie per valutare le richieste di autorizzazione.

Le entità vengono identificate attraverso il loro `UID` (Unique Identifier). Esso viene specificato tramite il namespace, il tipo e il nome dell'entità (racchiuso tra virgolette), separati da `":"`: `namespace::type::"name"`.

Un esempio di entità è `Application::User::"Cosimo"`. Il tipo delle azioni (actions) è sempre "Action".

L'utilizzo di identificatori che sono mutabili o riciclabili (il principale e la risorsa) deve essere assolutamente evitato per questioni di sicurezza: se ad esempio l'utente Cosimo cambiasse username, le politiche non funzionerebbero più. Addirittura, se un utente si registrasse con lo stesso username Cosimo dal momento che questo è tornato disponibile, allora questo utente avrebbe automaticamente l'autorizzazione per tutte le politiche definite per Cosimo in precedenza. Per questo, si dovrebbero sempre utilizzare gli UUID (Universally Unique Identifiers) autogenerabili (o sistemi equivalenti che risolvano il problema). L'eccezione va per actions, che sono stringhe sono immutabili (in quanto configurazioni a livello di sistema definite dal proprietario dell'applicazione). In questo lavoro si utilizza, a scopo illustrativo e per semplificare la comprensione, l'utilizzo di stringhe per identificare le entità.

Il file delle entità è un file JSON (JavaScript Object Notation) che viene passato al motore di autorizzazioni ogni qualvolta viene effettuata una richiesta. Il file contiene un set di entità (separate ognuna da una virgola) utilizzando le parentesi quadre. Ogni oggetto (che rappresenta un'entità) possiede tre elementi in un ordine qualsiasi: "uid", "parents" e "attrs", che devono essere "riempiti" in maniera opportuna come spiegato di seguito.

La struttura generale del file è quindi la seguente:

```
[
  {
    "uid": {...},
    "parents": [...],
    "attrs": {...}
  },
  ...,
  {
    "uid": {...},
    "parents": [...],
    "attrs": {...}
  }
]
```

uid

Lo uid identifica l'entità in questione e deve essere specificato il tipo e l'identificatore. Si prenda ad esempio l'entità User::"Cosimo"

```
"uid": {  
  "__entity": {  
    "type": "User",  
    "id": "Cosimo"  
  }  
}
```

oppure in maniera più compatta, ma solamente se è stato fornito lo schema che indica che le entità specificate sono presenti

```
"uid": {  
  "type": "User",  
  "id": "Cosimo"  
}
```

parents

Il campo parents identifica gli ascendenti della gerarchia dell'entità; questo è una lista JSON che può contenere più elementi (cioè un'entità può discendere da più gerarchie). La struttura è quindi la seguente:

```
"parents": [  
  {  
    "type": "Role",  
    "id": "Admin"  
  },  
  ...,  
  {  
    "type": "Group",  
    "id": "Developers"  
  }  
]
```

Anche in questo caso si devono inserire entità, quindi è possibile

inserire o omettere `--entity: {}`, sempre nel caso in cui lo schema sia stato fornito

attrs

Il campo `attrs` permette di specificare gli attributi dell'entità. Gli attributi sono rappresentati come quelli che abbiamo visto in Record in [3.4.1](#), dove la chiave è il nome dell'attributo e il valore è il valore dell'attributo. Gli attributi possono essere di qualsiasi tipo di dato supportato da Cedar. Un esempio è la seguente:

```
"attrs": {
  "username": "Cosimo",
  "phone": "1234567890",
  "age": 22
}
```

Tutti i tipi di dato di Cedar sono supportati, quindi è possibile inserire attributi di tipo Long, String, Boolean, Set, Record, Entity e Extension. Per quanto riguarda quest'ultimo, è possibile inserire attributi Extension decimal e ipaddr, ed è necessario specificarlo; si prenda come esempio l'attributo "gateway" di tipo ipaddr:

```
"gateway": {
  "--extn": {
    "fn": "ip",
    "arg": "192.168.1.1"
  }
}
```

oppure in maniera più compatta, ma sempre se è stato fornito lo schema che indica che le entità sono presenti

```
"gateway": {
  "fn": "ip",
  "arg": "192.168.1.1"
}
```

`fn` è il nome dell'extension, mentre `arg` è il suo valore.

3.4.6 *Policy Evaluation*

La "policy evaluation" è la fase in cui ogni qualvolta un utente esegue un'azione su una risorsa protetta, viene invocato il motore di autorizzazioni di Cedar per accettare (Allow) o negare (Deny) la richiesta. Una richiesta di autorizzazione è composta dagli elementi `principal`, `action`, `resource` e `context`. I primi tre sono identificativi di entità, mentre il contesto (`context`) è un record. Insieme a questi elementi viene fornita anche la lista delle entità con i loro attributi.

Il contesto viene utilizzato per fornire qualsiasi tipo di informazione aggiuntiva che è specifica di quella richiesta, come ad esempio la data e l'ora dell'invio della richiesta o l'indirizzo IP (un identificativo logico del dispositivo) del mittente. Gli attributi del contesto vengono richiamati tramite la "dot notation" (`context.attributeName`, dove `context` è fisso e `attributeName` indica il nome dell'attributo di interesse del contesto) e utilizzati all'interno delle clausole `when` e `unless` nelle politiche. La struttura di `context` è la stessa di quella vista in `attrs` nel file delle entità.

Come abbiamo visto in Sezione 3.3, il motore di autorizzazioni sceglie le politiche rilevanti, e per ognuna di esse controlla la soddisfaccibilità della richiesta; è possibile anche che la valutazione di una policy ritorni un errore. La decisione finale per la richiesta di autorizzazione si basa sul seguente algoritmo: se una qualunque delle politiche soddisfatte con effetto `forbid` viene valutata a `true`, allora la decisione finale risulta `Deny`; in caso contrario, se una qualunque delle politiche soddisfatte con effetto `permit` viene valutata a `true`, allora la decisione finale risulta `Allow`; se nessuna di queste due condizioni viene soddisfatta, allora vuol dire che nessuna politica è risultata soddisfatta, quindi la decisione finale è `Deny`.

Nel valutare una politica, il motore di autorizzazioni sostituisce a tutte le occorrenze di `principal`, `action`, `resource` e `context` nella politica i valori che ha recuperato dalla richiesta di autorizzazione. A quel punto vengono valutate le espressioni che si sono formate. In conclusione, se (in ordine) l'espressione del `principal`, quella di `action` e quella di `resource` vengono valutate tutte a `true`, tutte le condizioni all'interno della clausola `when` vengono valutate a `true` e tutte quelle all'interno della clausola `unless` vengono valutate a `false`, allora la politica viene soddisfatta e la valutazione finale di questa ritorna `true`. In caso contrario, questa ritorna `false`. Durante la valutazione della politica, se una di queste condizioni

risulta in un errore, allora la valutazione della politica si ferma e la politica stessa non viene soddisfatta.

3.5 BEST PRACTICES

Questa sezione è dedicata a fornire una serie di consigli e convenzioni generali.

- **Utilizzare nomi significativi per le entità** - È importante dare nomi significativi ai tipi delle entità e i loro attributi, in modo da rendere più chiaro il codice e facilitare la comprensione. È consigliabile utilizzare nomi univoci, in modo da evitare confusione e possibili errori.
- **Utilizzare nomi con uno stile consistente** - Utilizzare la notazione PascalCase (ogni parola inizia con una lettera maiuscola, senza spaziature di nessun tipo, tutti gli altri caratteri in minuscolo) per i nomi dei namespace e i tipi delle entità, e camelCase (come PascalCase ma con la primissima lettera in minuscolo) per i loro attributi, in modo da rendere più leggibile il codice e facilitarne la comprensione.
- **Utilizzare UUID per le entità** - È consigliabile utilizzare gli UUID (Universally Unique Identifiers) autogenerati per identificare le entità, come riportato in Sezione [3.4.5](#).
- **Utilizzare i template** - è consigliabile utilizzare i template per definire politiche generiche che possono essere riutilizzate in contesti diversi. I template permettono di evitare la ridondanza del codice e rendere più efficiente la scrittura delle politiche.
- **Definire lo schema ed effettuare la policy validation** - È consigliabile definire uno schema per le entità, le azioni e i tipi comuni utilizzati nelle politiche. Lo schema permette di garantire la correttezza delle politiche ed evitare errori durante la valutazione delle richieste di autorizzazione. Ogni qualvolta viene effettuata una modifica alle politiche è consigliabile attuare la validazione per minimizzare gli errori durante la valutazione delle richieste di autorizzazione.

- **Mantenere il codice indentato correttamente** - Inserire tabulazioni all'inizio di ogni riga all'interno di parentesi per rendere più leggibile il codice e facilitarne la comprensione.
- **Utilizzare i campi delle politiche in modo corretto** - Inserire le informazioni del soggetto in `principal`, quelle dell'azione in `action` e quelle della risorsa in `resource` evitando di utilizzare le clausole `when` e `unless` quando possibile, per consentire alle politiche di essere più comprensibili e permettere la scelta delle politiche rilevanti da parte dell'algoritmo di `policy-slicing`.
- **Controllare la presenza di attributi nelle politiche** - Assicurarsi, quando si definiscono espressioni che utilizzano attributi di entità, che questi siano presenti tramite l'operatore `has`, in modo da evitare errori durante la valutazione delle richieste di autorizzazione.
- **Utilizzare il campo `context` in maniera corretta** - Inserire nel campo `context` solo le informazioni specifiche di una richiesta di autorizzazione evitando le informazioni persistenti come quelle associate alle entità dell'applicazione.

ESPERIENZA APPLICATIVA

In questa parte della ricerca viene presentata l'applicazione del linguaggio Cedar e dei relativi strumenti software a un caso di studio inerente ad un'applicazione universitaria. È possibile visitare la repository GitHub del progetto in [8]. Si precisa che il progetto illustrato in questo capitolo è puramente fittizio e non è stato implementato. L'obiettivo è mostrare come si può utilizzare Cedar in un contesto reale. Alcune definizioni sono state inserite al solo scopo di mostrare l'utilizzo di specifici elementi del linguaggio. Il suo sviluppo è stato realizzato tramite l'applicazione **Visual Studio Code** e l'**estensione** per il supporto al linguaggio Cedar. La verifica delle richieste è stata effettuata utilizzando CedarCLI [12]. Si ricorda inoltre che nel seguente progetto non vengono utilizzati gli UUID per identificare le entità, ma semplici stringhe a scopo illustrativo.

Si supponga di essere in un contesto di un'applicazione universitaria dove gli studenti e i docenti possono effettuare una serie di azioni in base a varie condizioni. I docenti fanno parte di un gruppo in base al loro ruolo, in questo caso amministratori, ricercatori o insegnanti; gli amministratori hanno privilegi per l'esecuzione di alcune azioni, i ricercatori possono pubblicare i propri articoli per essere visualizzati dal pubblico. Gli studenti possono iscriversi ai corsi che desiderano se questi fanno parte della loro facoltà e se iscritti (cioè se hanno una matricola). Chiunque sia iscritto ad un corso può visualizzare tutti i partecipanti iscritti. I docenti responsabili di un corso (al solo scopo illustrativo) possono rimuovere gli studenti iscritti se questi hanno una media superiore al 28. Gli studenti possono rimuoversi dai corsi in qualsiasi momento. I corsi possono essere rimossi dai proprietari quando terminati e se nessuno vi è iscritto.

Di seguito un estratto dello schema in formato "Human-Readable", la cui versione completa si trova in appendice [A](#):

Il seguente codice mostra la definizione del tipo User, composto dai due attributi age e id, entrambi di tipo Long; id, che indica la matricola, è un attributo opzionale.

```
type User = {
  age: Long,
  id?: Long
};
```

L'entità Student è composta dagli attributi faculty di tipo String, weightedAverage di tipo decimal e userInformation, contenente le informazioni dal tipo User.

```
entity Student {
  userInformation: User,
  faculty: String,
  weightedAverage: decimal
};
```

L'entità Article è composta dagli attributi title e genre, entrambi di tipo String.

```
entity Article {
  title: String,
  genre: String
};
```

L'azione ReadArticle, facente parte dei gruppi di azioni ResearchDepartmentActions e ReadResource, viene applicata dalle entità di tipo Student e Professore sulle entità di tipo Article. È opzionale inserire un attributo booleano isPublic come context.

```
action ReadArticle in [ResearchDepartmentActions, ReadResource]
  appliesTo {
    principal: [Student, Professor],
    resource: Article,
    context: {
      isPublic?: Bool
    }
  }
```

```
}  
};
```

Un estratto dello schema in formato JSON degli stessi elementi, la cui versione completa si trova in appendice [B](#):

```
"commonTypes": {  
  "User": {  
    "type": "Record",  
    "attributes": {  
      "age": {  
        "type": "Long"  
      },  
      "id": {  
        "type": "Long",  
        "required": false  
      }  
    }  
  }  
},
```

```
  "Student": {  
    "shape": {  
      "type": "Record",  
      "attributes": {  
        "faculty": {  
          "type": "String"  
        },  
        "userInformation": {  
          "type": "User"  
        },  
        "weightedAverage": {  
          "type": "Extension",  
          "name": "decimal"  
        }  
      }  
    }  
  }  
},
```

```
  "Article": {  
    "shape": {
```

```

        "type": "Record",
        "attributes": {
            "genre": {
                "type": "String"
            },
            "title": {
                "type": "String"
            }
        }
    }
}

```

```

"ReadArticle": {
    "appliesTo": {
        "resourceTypes": [
            "Article"
        ],
        "principalTypes": [
            "Student",
            "Professor"
        ],
        "context": {
            "type": "Record",
            "attributes": {
                "isPublic": {
                    "type": "Boolean",
                    "required": false
                }
            }
        }
    },
    "memberOf": [
        {
            "id": "ReadResource"
        },
        {
            "id": "ResearchDepartmentActions"
        }
    ]
},

```

Di seguito un estratto del file delle entità, la cui versione completa si trova in appendice [C](#):

Lo studente "student1" con matricola 1 e di età 25 è iscritto alla facoltà di Computer Science e ha una media ponderata di 28. Non possiede entità ascendenti nella sua gerarchia.

```
{
  "uid": {
    "type": "Student",
    "id": "student1"
  },
  "attrs": {
    "userInformation": {
      "age": 25,
      "id": 1
    },
    "faculty": "Computer Science",
    "weightedAverage": {
      "fn": "decimal",
      "arg": "28.0"
    }
  },
  "parents": []
},
```

L'articolo "article3" di titolo "Renewable Energy Sources: A Comparative Study of Solar and Wind Power" è del genere Engineering e non possiede entità ascendenti nella sua gerarchia.

```
{
  "uid": {
    "type": "Article",
    "id": "article3"
  },
  "attrs": {
    "title": "Renewable Energy Sources: A Comparative Study of
      Solar and Wind Power",
    "genre": "Engineering"
  },
  "parents": []
}
```

Le politiche del progetto:

Questa politica permette allo studente "student3" di effettuare l'azione "SubscribeToCourse" sul corso "course1".

```
@id("policy0")
permit (
  principal == Student::"student3",
  action == Action::"SubscribeToCourse",
  resource == Course::"course1"
);
```

Questa politica permette a chiunque di effettuare le azioni "SubscribeToCourse" e "UnsubscribeToCourse" quando è presente l'attributo "id" su qualsiasi entità e la facoltà del principal corrisponde a quella della resource.

```
@id("policy1")
permit (
  principal,
  action in [Action::"SubscribeToCourse",
    Action::"UnsubscribeFromCourse"],
  resource
)
when
{ principal.userInformation has id && principal.faculty ==
  resource.faculty };
```

Questa politica permette a chiunque di visualizzare i partecipanti di un corso quando vi si è iscritti.

```
@id("policy2")
permit (
  principal,
  action == Action::"ReadParticipants",
  resource
)
when { context.isPrincipalSubscribed };
```

Questa politica permette a qualunque professore facente parte del gruppo "ProfessorTeam::admin" di effettuare una qualsiasi azione del

gruppo ReadResource su qualsiasi entità.

```
@id("policy3")
permit (
    principal in ProfessorTeam::"admin",
    action in Action::"ReadResource",
    resource
);
```

Questa politica permette ai professori facenti parte del gruppo "ProfessorTeam::researcher" di effettuare le azioni del gruppo "ResearchDepartmentActions" su qualsiasi entità.

```
@id("policy4")
permit (
    principal in ProfessorTeam::"researcher",
    action in Action::"ResearchDepartmentActions",
    resource
);
```

Questa politica vieta a chiunque di effettuare l'azione "PublishArticle" su qualsiasi entità quando l'indirizzo IP è "1.2.3.4" oppure l'indirizzo fornito è del tipo IPv6.

```
@id("policy5")
forbid (
    principal,
    action == Action::"PublishArticle",
    resource
)
when { context.sourceIp != ip("1.2.3.4") ||
    context.sourceIp.isIpv6() };
```

Questa politica permette a chiunque di effettuare l'azione "ReadArticle" su qualsiasi entità quando il principal è un professore, oppure la risorsa è pubblica.

```
@id("policy6")
permit (
    principal,
    action == Action::"ReadArticle",
```

```

    resource
  )
  when
  { if context has isPublic then context.isPublic else principal is
    Professor };

```

Questa politica vieta a chiunque di effettuare l'azione "ReadArticle" quando il titolo dell'articolo contiene le stringhe "Artificial Intelligence", "AI" oppure ha "Artificial Intelligence" come genere.

```

@id("policy7")
forbid (
  principal,
  action == Action::"ReadArticle",
  resource
)
when
{
  resource.title like "*Artificial Intelligence*" ||
  resource.title like "*AI*" ||
  resource.genre == "Artificial Intelligence"
};

```

Questa politica permette ai professori che gestiscono il corso in questione di effettuare l'azione "DeleteCourse" quando l'anno corrente è maggiore rispetto all'anno del corso e non ha studenti iscritti.

```

@id("policy8")
permit (
  principal,
  action == Action::"DeleteCourse",
  resource
)
when { principal in resource.professors && context.currentYear >
  resource.year }
unless { context.studentsSubscribedToCourse > 0 };

```

Questa politica permette ai professori di effettuare l'azione "RemoveStudent" da un corso del quale sono responsabili se la media dello studente non è minore di 28.

```
@id("policy9")
permit (
  principal,
  action == Action::"RemoveStudent",
  resource
)
when { principal in context.course.professors }
unless { resource.weightedAverage.lessThan(decimal("28.0")) };
```

Una volta definite le politiche della nostra applicazione, è possibile effettuare la loro validazione tramite lo schema definito in precedenza utilizzando Cedar CLI:

```
cedar validate -s cedarschema.json -p policies.cedar
```

Con Cedar CLI, è possibile anche effettuare una richiesta di autorizzazione e valutarne il risultato rispetto alle politiche definite in precedenza; si possono inserire i vari elementi della richiesta come argomenti, oppure utilizzare un file JSON contenente tutti gli elementi necessari per la richiesta stessa.

Il comando di seguito permette di valutare la richiesta di autorizzazione definita nel file "firstRequest.json" rispetto alle politiche definite nel file "policies.cedar", utilizzando lo schema "cedarschema.json" e il file delle entità "cedarentities.json". Il parametro "-v" permette di visualizzare i dettagli della richiesta e del risultato.

```
cedar authorize --request-json firstRequest.json -p policies.cedar
               -s cedarschema.json --entities cedarentities.json -v
```

Di seguito una lista di possibili richieste di autorizzazione con il loro risultato.

```
{
  "principal": "Student::\"student3\"",
  "action": "Action::\"SubscribeToCourse\"",
  "resource": "Course::\"course1\"",
  "context": {}
}
```

ALLOW

note: this decision was due to the following policies:
policy0

La decisione finale di questa richiesta è "Allow", grazie alla politica con identificativo "policy0".

Per le seguenti richieste di autorizzazione la prassi è analoga, basta cambiare il nome del file JSON della richiesta.

```
{
  "principal": "Student::\"student1\"",
  "action": "Action::\"SubscribeToCourse\"",
  "resource": "Course::\"course2\"",
  "context": {}
}
```

DENY

note: no policies applied to this request

```
{
  "principal": "Student::\"student1\"",
  "action": "Action::\"SubscribeToCourse\"",
  "resource": "Course::\"course1\"",
  "context": {}
}
```

ALLOW

note: this decision was due to the following policies:
policy1

```
{
  "principal": "Student::\"student1\"",
  "action": "Action::\"ReadParticipants\"",
  "resource": "Course::\"course1\"",
}
```



```
"context": {  
  "isPrincipalSubscribed": true  
}  
}
```

ALLOW

note: this decision was due to the following policies:
policy2

```
{  
  "principal": "Student::\"student1\"",  
  "action": "Action::\"UnsubscribeFromCourse\"",  
  "resource": "Course::\"course1\"",  
  "context": {}  
}
```

ALLOW

note: this decision was due to the following policies:
policy1

```
{  
  "principal": "Student::\"student1\"",  
  "action": "Action::\"ReadParticipants\"",  
  "resource": "Course::\"course1\"",  
  "context": {  
    "isPrincipalSubscribed": false  
  }  
}
```

DENY

note: no policies applied to this request

```
{  
  "principal": "Professor::\"professor1\"",  
  "action": "Action::\"PublishArticle\"",
```

```

    "resource": "Article::\"article3\"",
    "context": {
      "sourceIp": {
        "fn": "ip",
        "arg": "1.2.3.4"
      }
    }
  }
}

```

ALLOW

note: this decision was due to the following policies:
policy4

```

{
  "principal": "Professor::\"professor3\"",
  "action": "Action::\"PublishArticle\"",
  "resource": "Article::\"article1\"",
  "context": {
    "sourceIp": {
      "fn": "ip",
      "arg": "1.2.3.4"
    }
  }
}

```

DENY

note: no policies applied to this request

```

{
  "principal": "Professor::\"professor3\"",
  "action": "Action::\"ReadArticle\"",
  "resource": "Article::\"article1\"",
  "context": {}
}

```

DENY

note: this decision was due to the following policies:
policy7

```
{
  "principal": "Professor::\"professor2\"",
  "action": "Action::\"DeleteCourse\"",
  "resource": "Course::\"course1\"",
  "context": {
    "currentYear": 2024,
    "studentsSubscribedToCourse": 0
  }
}
```

DENY

note: no policies applied to this request

```
{
  "principal": "Professor::\"professor1\"",
  "action": "Action::\"DeleteCourse\"",
  "resource": "Course::\"course1\"",
  "context": {
    "currentYear": 2024,
    "studentsSubscribedToCourse": 4
  }
}
```

DENY

note: no policies applied to this request

```
{
  "principal": "Professor::\"professor1\"",
  "action": "Action::\"DeleteCourse\"",
  "resource": "Course::\"course1\"",
  "context": {
    "currentYear": 2024,
    "studentsSubscribedToCourse": 0
  }
}
```

```

    }
  }

```

ALLOW

note: this decision was due to the following policies:
policy8

```

{
  "principal": "Professor::\"professor1\"",
  "action": "Action::\"RemoveStudent\"",
  "resource": "Student::\"student1\"",
  "context": {
    "course": {
      "type": "Course",
      "id": "course1"
    }
  }
}

```

ALLOW

note: this decision was due to the following policies:
policy9

```

{
  "principal": "Professor::\"professor3\"",
  "action": "Action::\"ReadArticle\"",
  "resource": "Article::\"article3\"",
  "context": {}
}

```

ALLOW

note: this decision was due to the following policies:
policy3
policy6

```
{
  "principal": "Professor::\"professor1\"",
  "action": "Action::\"PublishArticle\"",
  "resource": "Article::\"article2\"",
  "context": {
    "sourceIp": {
      "fn": "ip",
      "arg": "192.168.1.11"
    }
  }
}
```

DENY

note: this decision was due to the following policies:
policy5

CONCLUSIONI

Questo elaborato ha presentato Cedar, un linguaggio sviluppato dal team di AWS che permette di definire politiche di controllo degli accessi in maniera dichiarativa. È stata descritta la sintassi e la semantica del linguaggio con un esempio concreto dell'utilizzo di Cedar per definire il controllo degli accessi in un'applicazione generica.

Cedar è open-source: chiunque può contribuire al suo sviluppo. Questo permette di avere un'ampia comunità di sviluppatori che possono migliorare le funzionalità e risolvere eventuali bug. Alcune delle modifiche suggerite dalla community di Cedar che potrebbero risultare utili per migliorare il linguaggio sono:

- Capacità di editing avanzate: implementazione di funzionalità di editing avanzate come autocompletamento intelligente in strumenti di editing come Visual Studio Code, per il quale è stata creata un'estensione per il supporto al linguaggio. Questo renderebbe la scrittura di codice più efficiente e ridurrebbe la possibilità di errori.
- Compatibilità con più linguaggi: rendere Cedar utilizzabile con una varietà di linguaggi di programmazione. Sono già stati sviluppati bindings che permettono a Cedar di essere chiamato da altri linguaggi come Java. L'ampliamento su questo fronte aumenterebbe notevolmente la flessibilità e l'applicabilità del linguaggio.
- Compressione delle politiche: il mantenimento delle politiche il più compatte possibile per minimizzare l'utilizzo della memoria. La maggior parte delle politiche iniziano con gli stessi caratteri: per i file contenenti più di 50 politiche, il risparmio di spazio sarebbe

superiore all'80%. Un approccio per ridurre le dimensioni delle politiche è la pre-analisi e precompilazione di esse in una rappresentazione in byte-code con l'effetto sia di ridurre le dimensioni complessive, sia di abbassare i costi di valutazione. Pertanto, se si presentassero scenari futuri di questo tipo ci sarebbero vantaggi per chiunque desideri ridurre i costi di valutazione, indipendentemente dal numero di politiche.

Alcuni di questi suggerimenti sono già in fase di sviluppo e il team di Cedar è sempre alla ricerca di feedback e suggerimenti per migliorare il linguaggio e renderlo più utile per la community.

In [3] vengono comparate le performance di Cedar con quelle dei linguaggi OpenFGA [2] e Rego [6], prendendo in considerazione il tempo di risposta di 100000 richieste di autorizzazione generate in maniera randomica su tre applicazioni diverse:

1. Semplificazione di Google Drive (gdrive)
2. Semplificazione di GitHub (github)
3. Gestore di attività da completare (TinyTodo)

Considerando i risultati dei test effettuati dal team di Cedar, questo linguaggio risulta rispettivamente 28.7, 34.4, e 35.2 più veloce di OpenFGA nelle applicazioni gdrive, github e TinyTodo; rispetto a Rego è 60.4, 80.8 e 42.8 più veloce. All'aumentare del numero delle entità e delle politiche, la differenza di performance diventa sempre più marcata rispetto alla concorrenza. Questi risultati mostrano Cedar preferibile rispetto agli altri linguaggi.

Il presente lavoro si conclude suggerendo possibili sviluppi per chi è interessato ad analizzare l'argomento. È interessante esplorare la parte applicativa con Rust e approfondire gli aspetti formali del linguaggio e dell'utilizzo del ragionamento automatico con Dafny e differential testing, oppure esaminare in dettaglio l'implementazione dell'algoritmo di policy-slicing. Essendo Cedar un linguaggio recente e in continua evoluzione, vi sono interessanti opportunità di sviluppo e miglioramento e nuove funzionalità.

BIBLIOGRAFIA

- [1] Inc. Amazon Web Services. Cedar policy language. <https://www.cedarpolicy.com/en>.
- [2] Auth0. Openfga. <https://openfga.dev/>.
- [3] Joseph Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Elefterios Ioannidis, John Kastner, Anwar Mamat, et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization (extended version). *arXiv preprint arXiv:2403.04651*, 2024.
- [4] CyLab. Mike hicks - "cedar: A language for expressing fast, safe, and fine-grained authorization policies". <https://www.youtube.com/watch?v=hdc5JZZNDSs>.
- [5] Softwave Technology for Business. Amazon web services - cos'è e come funziona? <https://www.softwave-soltec.it/amazon-web-service-cose-e-come-funziona/>.
- [6] Cloud Native Computing Foundation. Rego policy language. <https://www.openpolicyagent.org/docs/latest/policy-language/>.
- [7] Mike Hicks. How we built cedar with automated reasoning and differential testing. <https://www.amazon.science/blog/how-we-built-cedar-with-automated-reasoning-and-differential-testing>.
- [8] Cosimo Matassini. University cedar experience. <https://github.com/CosimoMatassini/university-cedar-experience>, 2024.
- [9] Darin McAdams. Cedarland blog. <https://cedarland.blog/>.
- [10] Microsoft. Cos'è il controllo degli accessi? <https://www.microsoft.com/it-it/security/business/security-101/what-is-access-control>.

- [11] Graham Neray. The 10 types of authorization: The families of rbac, rebac and abac. <https://www.osohq.com/post/ten-types-of-authorization>, February 2024.
- [12] Amazon Web Services. Cedar cli. <https://crates.io/crates/cedar-policy-cli>.
- [13] Amazon Web Services. Cedar policy language in action. <https://catalog.workshops.aws/cedar-policy-language-in-action/en-US/20-tutorial>.
- [14] Amazon Web Services. Cosa è il ragionamento automatico? <https://aws.amazon.com/it/what-is/automated-reasoning/>.



SCHEMA HUMAN-READABLE

```
type User = {
  age: Long,
  id?: Long
};

entity ProfessorTeam;

entity Student {
  userInformation: User,
  faculty: String,
  weightedAverage: decimal
};

entity Professor in ProfessorTeam {
  userInformation : User
};

entity Course {
  faculty: String,
  year: Long,
  professors: Set<Professor>
};

entity Article {
  title: String,
  genre: String
};

action ReadResource, ResearchDepartmentActions;
```

```

action ReadParticipants in ReadResource appliesTo {
  principal: [Student, Professor],
  resource: Course,
  context: {
    isPrincipalSubscribed: Bool
  }
};

```

```

action SubscribeToCourse, UnsubscribeFromCourse appliesTo {
  principal: Student,
  resource: Course
};

```

```

action RemoveStudent appliesTo {
  principal: Professor,
  resource: Student,
  context: {
    course: Course
  }
};

```

```

action DeleteCourse appliesTo {
  principal: Professor,
  resource: Course,
  context: {
    currentYear: Long,
    studentsSubscribedToCourse: Long
  }
};

```

```

action PublishArticle in ResearchDepartmentActions appliesTo {
  principal: Professor,
  resource: Article,
  context: {
    sourceIp: ipaddr
  }
};

```

```

action ReadArticle in [ResearchDepartmentActions, ReadResource]
  appliesTo {
    principal: [Student, Professor],

```

```
resource: Article,  
context: {  
  isPublic?: Bool  
}  
};
```


B

SCHEMA JSON

```
{
  "": {
    "commonTypes": {
      "User": {
        "type": "Record",
        "attributes": {
          "age": {
            "type": "Long"
          },
          "id": {
            "type": "Long",
            "required": false
          }
        }
      }
    },
    "entityTypes": {
      "ProfessorTeam": {},
      "Student": {
        "shape": {
          "type": "Record",
          "attributes": {
            "faculty": {
              "type": "String"
            },
            "userInformation": {
              "type": "User"
            },
            "weightedAverage": {
```

```

        "type": "Extension",
        "name": "decimal"
    }
}
},
"Professor": {
    "memberOfTypes": [
        "ProfessorTeam"
    ],
    "shape": {
        "type": "Record",
        "attributes": {
            "userInformation": {
                "type": "User"
            }
        }
    }
},
"Course": {
    "shape": {
        "type": "Record",
        "attributes": {
            "faculty": {
                "type": "String"
            },
            "professors": {
                "type": "Set",
                "element": {
                    "type": "Entity",
                    "name": "Professor"
                }
            },
            "year": {
                "type": "Long"
            }
        }
    }
},
"Article": {
    "shape": {
        "type": "Record",

```



```

        "attributes": {
            "genre": {
                "type": "String"
            },
            "title": {
                "type": "String"
            }
        }
    },
    "actions": {
        "ReadParticipants": {
            "appliesTo": {
                "resourceTypes": [
                    "Course"
                ],
                "principalTypes": [
                    "Student",
                    "Professor"
                ],
                "context": {
                    "type": "Record",
                    "attributes": {
                        "isPrincipalSubscribed": {
                            "type": "Boolean"
                        }
                    }
                }
            },
            "memberOf": [
                {
                    "id": "ReadResource"
                }
            ]
        },
        "RemoveStudent": {
            "appliesTo": {
                "resourceTypes": [
                    "Student"
                ],
                "principalTypes": [

```

```

        "Professor"
    ],
    "context": {
        "type": "Record",
        "attributes": {
            "course": {
                "type": "Entity",
                "name": "Course"
            }
        }
    }
},
"DeleteCourse": {
    "appliesTo": {
        "resourceTypes": [
            "Course"
        ],
        "principalTypes": [
            "Professor"
        ],
        "context": {
            "type": "Record",
            "attributes": {
                "currentYear": {
                    "type": "Long"
                },
                "studentsSubscribedToCourse": {
                    "type": "Long"
                }
            }
        }
    }
},
"PublishArticle": {
    "appliesTo": {
        "resourceTypes": [
            "Article"
        ],
        "principalTypes": [
            "Professor"
        ],

```

```
    "context": {
      "type": "Record",
      "attributes": {
        "sourceIp": {
          "type": "Extension",
          "name": "ipaddr"
        }
      }
    },
    "memberOf": [
      {
        "id": "ResearchDepartmentActions"
      }
    ],
    "ReadResource": {
      "appliesTo": {
        "resourceTypes": [],
        "principalTypes": []
      }
    },
    "ResearchDepartmentActions": {
      "appliesTo": {
        "resourceTypes": [],
        "principalTypes": []
      }
    },
    "SubscribeToCourse": {
      "appliesTo": {
        "resourceTypes": [
          "Course"
        ],
        "principalTypes": [
          "Student"
        ]
      }
    },
    "ReadArticle": {
      "appliesTo": {
        "resourceTypes": [
          "Article"
        ]
      }
    }
  }
}
```

```

    ],
    "principalTypes": [
        "Student",
        "Professor"
    ],
    "context": {
        "type": "Record",
        "attributes": {
            "isPublic": {
                "type": "Boolean",
                "required": false
            }
        }
    }
},
"memberOf": [
    {
        "id": "ReadResource"
    },
    {
        "id": "ResearchDepartmentActions"
    }
]
},
"UnsubscribeFromCourse": {
    "appliesTo": {
        "resourceTypes": [
            "Course"
        ],
        "principalTypes": [
            "Student"
        ]
    }
}
}
}
}
}

```

IL FILE DELLE ENTITÀ

```
[
  {
    "uid": {
      "type": "ProfessorTeam",
      "id": "researcher"
    },
    "attrs": {},
    "parents": []
  },
  {
    "uid": {
      "type": "ProfessorTeam",
      "id": "teacher"
    },
    "attrs": {},
    "parents": []
  },
  {
    "uid": {
      "type": "ProfessorTeam",
      "id": "admin"
    },
    "attrs": {},
    "parents": []
  },
  {
    "uid": {
      "type": "Student",
      "id": "student1"
    }
  }
]
```

```

    },
    "attrs": {
      "userInformation": {
        "age": 25,
        "id": 1
      },
      "faculty": "Computer Science",
      "weightedAverage": {
        "fn": "decimal",
        "arg": "28.0"
      }
    },
    "parents": []
  },
  {
    "uid": {
      "type": "Student",
      "id": "student3"
    },
    "attrs": {
      "userInformation": {
        "age": 23
      },
      "faculty": "Physics",
      "weightedAverage": {
        "fn": "decimal",
        "arg": "0.0"
      }
    },
    "parents": []
  },
  {
    "uid": {
      "type": "Professor",
      "id": "professor1"
    },
    "attrs": {
      "userInformation": {
        "age": 45,
        "id": 3
      }
    }
  },

```

```

    "parents": [
      {
        "type": "ProfessorTeam",
        "id": "researcher"
      }
    ]
  },
  {
    "uid": {
      "type": "Professor",
      "id": "professor2"
    },
    "attrs": {
      "userInformation": {
        "age": 36,
        "id": 4
      }
    },
    "parents": [
      {
        "type": "ProfessorTeam",
        "id": "teacher"
      }
    ]
  },
  {
    "uid": {
      "type": "Professor",
      "id": "professor3"
    },
    "attrs": {
      "userInformation": {
        "age": 57,
        "id": 5
      }
    },
    "parents": [
      {
        "type": "ProfessorTeam",
        "id": "admin"
      }
    ]
  }
]

```

```

    },
    {
      "uid": {
        "type": "Course",
        "id": "course1"
      },
      "attrs": {
        "faculty": "Computer Science",
        "year": 2022,
        "professors": [
          {
            "type": "Professor",
            "id": "professor1"
          },
          {
            "type": "Professor",
            "id": "professor3"
          }
        ]
      },
      "parents": []
    },
    {
      "uid": {
        "type": "Course",
        "id": "course2"
      },
      "attrs": {
        "faculty": "Physics",
        "year": 2024,
        "professors": [
          {
            "type": "Professor",
            "id": "professor2"
          }
        ]
      },
      "parents": []
    },
    {
      "uid": {
        "type": "Article",

```



```

        "id": "article1"
    },
    "attrs": {
        "title": "The impact of AI on the future of work",
        "genre": "Artificial Intelligence"
    },
    "parents": []
},
{
    "uid": {
        "type": "Article",
        "id": "article2"
    },
    "attrs": {
        "title": "The Impact of Social Media on Adolescent Mental
            Health",
        "genre": "Humanities"
    },
    "parents": []
},
{
    "uid": {
        "type": "Article",
        "id": "article3"
    },
    "attrs": {
        "title": "Renewable Energy Sources: A Comparative Study of
            Solar and Wind Power",
        "genre": "Engineering"
    },
    "parents": []
}
]

```